

Scientific Communication, Collaboration and Design in Quantitative Research Using R

Daniel Hammarström

2025-09-05

Table of contents

Preface	4
I Data visualization	6
1 A grammar of graphics	8
1.0.1 Building blocks of a plot	8
2 Before plotting	12
2.0.1 Reading data into R	14
2.0.2 The verbs of data wrangling	15
2.0.3 From wide to long and back again	24
2.0.4 Joining data sets	29
2.0.5 Filters and sorting rows	30
2.0.6 Summaries	35
2.0.7 Arranging data frames	37
3 More advanced features of ggplot2	39
3.0.1 Labels and themes	46
4 Scientific graphic design	55
4.0.1 Revise and edit	55
4.1 Edit and revise in practice	70
4.2 Additional notes, tips and topics	71
4.2.1 Labels, annotations and special texts	71
5 Glamorous graphics	74
II Version control and collaborative scientific coding	76
6 Introduction to git and GitHub	78
6.0.1 Accounts and git-to-GitHub integration	78

6.0.2	Local git edits	79
6.1	Local and online git usage	86
6.1.1	Starting with GitHub	86
6.1.2	Starting with a RStudio project	87
6.2	Local to remote workflows	88
6.2.1	Branches and pull requests	91
6.2.2	Forks and pull requests	92
6.3	Finding help on the command line	93
6.4	More features in GitHub: Issues and Pages . . .	93
7	A tentative workflow for scientific writing	94
7.1	A basic template for an effective repository . . .	94
7.2	A workflow for solo writing	97
7.2.1	Large revisions - Branches	98
7.3	Writing in a team	98
7.3.1	Branches and pull requests	99
7.4	Doing formal testing	100
7.5	Going further - Make files	101
8	Writing a paper using Quarto	102
8.1	The quarto publishing system	102
8.2	The basics - What Quarto does	102
	References	104

Preface

Note

These course notes are under development. They will serve as the basis of the course *Scientific Communication, Collaboration and Design in Quantitative Research Using R* given as an elective in the PhD program Health and Welfare at the [University of Inland Norway](#).

The purpose of this collection of workshops is to provide basic skills and understanding of scientific visualization, collaboration, and thinking. These concepts are often not taught explicitly in postgraduate courses. Instead, they are assumed to be known and understood or somehow magically infused in students through their respective scientific environments. Instead of relying on magic, professional skills should be taught and developed continuously, explicitly, and systematically.

The first workshop deals with data visualization which is a fundamental skill for any scientist. Data visualization, like writing, serves multiple purposes in day-to-day scientific practice. Making graphs before data collection can help the researcher better formulate the scientific problem or explain core concepts for oneself, committees or funding agencies. Visualizations will also be important for exploratory data analysis, for discovering problematic data formatting or important patterns related to the scientific problem. Lastly, visualizations will aid in communicating scientific results to other researchers and the public.

Two workshops are devoted to collaboration in data-intensive science. The first workshop deals with the basics of version control and the second workshop deals with structuring collected data into data packages for analysis and sharing. Version control is a way to keep track of changes in a set of files. It also provides opportunities for effective collaboration in complex

projects. Using the R package system for data management and sharing is a way to ensure that data is well documented and downstream analyses are reproducible.

The last workshop deals with how we can develop scientific thinking using scientific models and data simulations. We will use graphical models to draw out assumptions and explore simulations as a way to test the robustness of our models and plan experiments.

Part I

Data visualization

The most apparent purpose of data visualizations in scientific contexts is to convey information not suited for text or tables. Such information may be comparisons between categories, relationships among variables, or trends over time (Gelman, Passarica, and Dodhia 2002; Tufte 2001). We often think of the scientific graph as the end product of our labor. However, data visualization can be an effective tool for thinking about scientific problems and performing exploratory data analysis before preparing a final report. These steps will likely take up more time than polishing *Figure 1* of your upcoming paper.

Data visualization is thus a skill, like writing, with several purposes. You may write notes for yourself and write to communicate with others. Notes come in many forms, just as messages you write to others. Similarly, data visualizations can have different purposes, such as

- Checking that your data is what could be expected
- Diagnosing statistical models
- Exploring relationships between many variables
- Exploring if your measurements are reliable and valid
- Conveying the key message from your study

Traditionally, and in many software implementations, data visualization is a matter of choosing from a menu of charts such as box plots, scatter plots, or line graphs. These can be very effective. However, you would not restrict your writing to pre-made templates. As a professional scientist in a quantitative genre, you need a toolbox for data visualizations that does not invoke limitations. By using a *grammar of graphics* you can be more creative in designing your visualizations.

1 A grammar of graphics

Building on Wilkinsons *The Grammar of Graphics* (Wilkinson and Wills 2005), Wickham (Wickham 2010, 2016) implemented a graphical grammar to R through the package `ggplot2`. The idea behind both Wickhams and Wilkinsons formalized syntax for creating data visualizations is to avoid special cases or duplicate methods and allow for the creation of data visualizations based on a set of core components used in all graphics (Wickham 2010; Wilkinson and Wills 2005).

The grammar (in the version described by Wickham (Wickham 2010)) has seven components that together create a `ggplot2` visualization (Figure 1.1).

The underlying *data* contains the variables that are *mapped to aesthetics* such as coordinates, colors, or shapes. Each aesthetic can be controlled through *scales* by assigning values to coordinates, colors, shapes, etc. *Geometric objects* create the visual representations of the mapping. Sometimes small multiples of the same graph are created using *facets*, which creates a subdivision of the data to be plotted in different panels. *Statistical transformations* create summaries of the data, however, in its simplest form, the transformation is the identity transformation. All graphical representations are plotted in a *coordinate* system. *Theme and annotations* adds non-data layers to the plot, which can include geometric shapes or text (Figure 1.1).

1.0.1 Building blocks of a plot

Using a similar example as in (Wickham 2010) a basic plot can be broken down into the data (see Table 1.1), geometric objects, scales and coordinates and annotations. These components, that can be individually manipulated, together creates the final plot (see Figure 1.2 a-d).

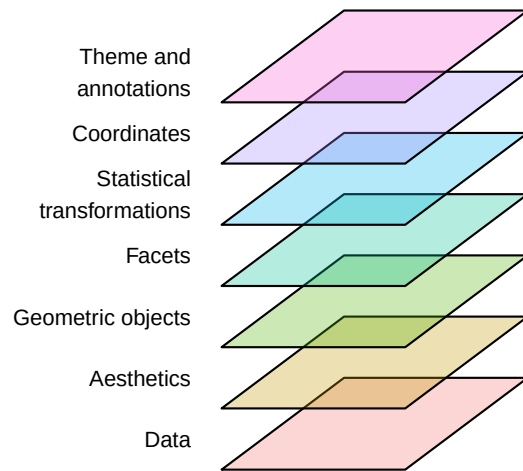


Figure 1.1: Components of a ggplot2 visualization

Table 1.1: Example data for a ggplot

x	y	shape
1.0	3.0	a
2.4	2.0	a
4.0	5.0	b
5.2	7.8	b

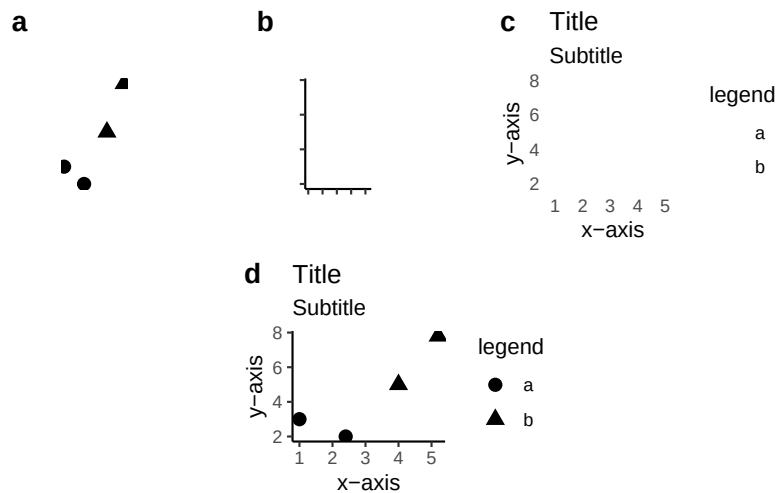


Figure 1.2: Three components, geometric objects (a), coordinates and scales (b) and plot annotations (c) are constituents of the complete plot (d)

The corresponding code to create this plot is fairly straight forward:

```
library(ggplot2)

df <- data.frame(x = c(1, 2.4, 4, 5.2),           ①
                 y = c(3, 2, 5, 7.8),
                 shape = c("a", "a", "b", "b"))

ggplot(data = df,                                ②
       mapping = aes(x = x, y = y, shape = shape)) + ③

geom_point(color = "black", size = 3) +          ④

labs(title = "Title",                             ⑤
     subtitle = "Subtitle",
     x = "x-axis",
     y = "y-axis",
     shape = "legend") +
theme_classic()                                  ⑥
```

- ① A simple data frame is saved containing the variables we will plot
- ② The data is explicitly called
- ③ Variables are mapped to coordinates (x and y) and shapes.
- ④ Geometric objects (points) are specified with color and size determined in the function
- ⑤ Annotations are added to the plot through the `labs` (labels) function.
- ⑥ A pre-built theme is used to change the default appearance of the plot.

2 Before plotting

As we saw above, a basic visualization can be created from a dataset represented in R as a data frame, which is the most common representation of data in R. A *tidy* data set has one observation per row and one variable per column. A tidy data set makes data visualization easy. However, not all data sets are friendly. In fact, some might be unfriendly because they are unhappy¹.

A lot of effort goes into making data sets suitable for visualization or statistical modelling. The good news is that R is especially suited for the process of importing and wrangling data. As with other common tasks in R there are numerous ways of achieving the same goals. This is a good thing because it allows for solutions to a wide range of problems. It is also a bad thing because it makes it difficult to getting started. A collection of R packages called the [Tidyverse](#) makes the process of getting started with data wrangling easier.

Tidyverse can be thought of as a dialect of the R language. The dialect is designed to make it easy to write sequential operations in a way that translates thoughts and ideas to code. Sequential operations are enabled by a pipe operator. Using a pipe operator we can call functions in sequential order to do specific operations on the data. We can write such a pipe as demonstrated below with the corresponding English language descriptions to the left.

Take the data ***then do*** filter the data based on x larger than 10 ***then do*** add a new calculated variable $z = x + y$ ***then do*** show the output

¹The reference to happiness is a reference to Wickham (2014) wherein Tolstoy’s Anna Karenina is quoted; “Happy families are all alike; every unhappy family is unhappy in its own way.” The Anna Karenina principle applies to data as non-tidy data can be non-tidy in many ways, tidy data however are tidy because they all share a common set of features.

```
data |>
  filter(x > 10) |>
  mutate(z = x + y) |>
  print()
```

The pipe operator (`|>`) takes any input and place it as the first argument in the following function². This is the mechanism that makes sequential operations possible. A pipe operator makes code more readable, consider the following two alternatives:

```
print(mutate(filter(select(data, var1:var3), var1 == "xxx"), var4 = var1 + var2)) ①
```

```
data |>
  select(var1:var3) |>
  filter(var1 == "xxx") |>
  mutate(var4 = var1 + var2) ②
```

- ① Alternative 1: A number of operations are performed on data, we have to read from in to out to see each step.
- ② Alternative 2: The same steps are performed as in alternative 1, in the same order.

The pipe in alternative 2 are structured with the pipe operator making the code more readable and easier to edit. Notice that the same functions are used in both cases.

In R there are two main pipe operators. We have the “base R” pipe, `|>`. This pipe operator is included in the base installation of R and available without loading any packages on start up. A second pipe belongs to the **magrittr** package. The **magrittr** pipe (`%>%`) has the same basic functionality as the base pipe; the left hand input is inserted as the first argument in any right hand function.

²A function is a basic building block of your R code. Function are designed for specific tasks and can be part of packages or created by the user. A function may take arguments such as `fun(<ARGUMENT1> = <default input>, <ARGUMENT2> = <default input>)`. Arguments can be used without explicitly naming them by putting expected input in at the right position. Orders of argument may be changed is argument names are used.

Sometimes you might want to place your input somewhere else than as the first argument in a right hand function. A placeholder can be used to indicate where you would like to place your input. The base pipe differs from the `magrittr` pipe in what symbol indicates a placeholder. In the example below, the function `some_fun()` expects data as the third argument. We need to use our placeholder to put the data in the correct position:

```
data |>
  some_fun(arg1 = c(1, 2),
           arg2 = "some.setting",
           arg3 = _)

library(tidyverse)
data %>%
  some_fun(arg1 = c(1, 2),
           arg2 = "some.setting",
           arg3 = .)
```

- ① With the base R forward pipe operator.
- ② With the `magrittr` forward pipe operator.

2.0.1 Reading data into R

Three packages makes reading tabular data into R easy. `readr` provides functions for reading and writing delimiter separated files, such as `.csv` or `.tsv`. `readxl` provides functions that imports data from excel files. An finally, `googlesheets4` makes it possible to read tabular data created in google sheets.

Data can also be loaded from packages in R since storing data is a convenient way of sharing. By including data in a package you are nudged to do some quality checks and document it. We will talk more about data packages in a later workshop.

2.0.2 The verbs of data wrangling

Once data is available in our workspace we will be able to wrangle it. In the examples below I will use a data set containing results from dual x-ray absorptiometry measurements available in the `exscidata` package. To install the `exscidata` package:

```
library(remotes)
install_github("dhammarstrom/exscidata")
```

The `dplyr` package provides a collection of verbs to facilitate wrangling. As mentioned above, “pipeable” functions takes a data frame as its first argument. This means that we can line up functions in sequential order using a pipe operator. In all verb functions, following the data argument follows a set of arguments that specifies what we wish to do with the data. The result of the operations performed by the function are returned as a data frame.

`dplyr` contains the following main data verbs:

- `select`
- `rename`
- `relocate`
- `mutate`
- `filter`
- `arrange`
- `summarize`

In addition, several helper function will aid our wrangling endeavors.

`dplyr` is loaded as part of the `tidyverse` package:

```
library(tidyverse)
```

2.0.2.1 Select, rename and relocate variables

Variables in a data frame may be selected and renamed. Such operation may have multiple purposes such as giving you a better overview of the data of interest or limiting what data to

display in a table. Renaming can make life easier if the data set contains long variable names.

Below we will store a subset of the data in a new data set. But we will first have a look at what column names are available:

```
library(exscidata) ①
glimpse(exscidata::dxadata) ②
```

- ① Loading the `exscidata` package
- ② Using `glimpse` we will get a overview of all available variables in the data set. The double `::` means that we are looking for the data set `dxadata` inside the package `exscidata`.

```
Rows: 80
Columns: 59
$ participant <chr> "FP28", "FP40", "FP21", "FP34", "FP23", "FP26", "FP36~
$ time <chr> "pre", "pre", "pre", "pre", "pre", "pre", "pre", "pre~
$ multiple <chr> "L", "R", "R", "R", "R", "R", "L", "R", "R", "L", "L"~
$ single <chr> "R", "L", "L", "L", "L", "L", "R", "L", "L", "R", "R"~
$ sex <chr> "female", "female", "male", "female", "male", "female~
$ include <chr> "incl", "incl", "incl", "incl", "incl", "excl", "incl~
$ age <dbl> 24.5, 22.1, 26.8, 23.1, 24.8, 24.2, 20.5, 20.6, 37.4,~
$ height <dbl> 170.0, 175.0, 184.0, 164.0, 176.5, 163.0, 158.0, 181.~
$ weight <dbl> 66.5, 64.0, 85.0, 53.0, 68.5, 56.0, 60.5, 83.5, 65.0,~
$ BMD.head <dbl> 2.477, 1.916, 2.306, 2.163, 2.108, 2.866, 1.849, 2.21~
$ BMD.arms <dbl> 0.952, 0.815, 0.980, 0.876, 0.917, 0.973, 0.871, 0.91~
$ BMD.legs <dbl> 1.430, 1.218, 1.598, 1.256, 1.402, 1.488, 1.372, 1.42~
$ BMD.body <dbl> 1.044, 0.860, 1.060, 0.842, 0.925, 0.984, 0.923, 1.01~
$ BMD.ribs <dbl> 0.770, 0.630, 0.765, 0.636, 0.721, 0.737, 0.648, 0.70~
$ BMD.pelvis <dbl> 1.252, 1.078, 1.314, 1.044, 1.154, 1.221, 1.194, 1.32~
$ BMD.spine <dbl> 1.316, 0.979, 1.293, 0.899, 1.047, 1.089, 1.006, 1.14~
$ BMD.whole <dbl> 1.268, 1.082, 1.325, 1.119, 1.181, 1.350, 1.166, 1.24~
$ fat.left_arm <dbl> 1168, 715, 871, 610, 788, 372, 932, 1312, 388, 668, 5~
$ fat.left_leg <dbl> 4469, 4696, 3467, 3023, 3088, 2100, 4674, 5435, 1873,~
$ fat.left_body <dbl> 6280, 4061, 7740, 3638, 6018, 2328, 4896, 9352, 2921,~
$ fat.left_whole <dbl> 12365, 9846, 12518, 7565, 10259, 5048, 10736, 16499, ~
$ fat.right_arm <dbl> 1205, 769, 871, 610, 741, 374, 940, 1292, 413, 716, 5~
$ fat.right_leg <dbl> 4497, 4900, 3444, 3017, 3254, 2082, 4756, 5455, 1782,~
$ fat.right_body <dbl> 6082, 3923, 8172, 3602, 5699, 2144, 4705, 8674, 2640,~
```



```

$ fat.right_whole <dbl> 12102, 9862, 12856, 7479, 10020, 4821, 10806, 15876, ~
$ fat.arms <dbl> 2373, 1484, 1742, 1220, 1529, 747, 1872, 2604, 802, 1~
$ fat.legs <dbl> 8965, 9596, 6911, 6040, 6342, 4182, 9430, 10890, 3655~
$ fat.body <dbl> 12362, 7984, 15912, 7239, 11717, 4472, 9601, 18026, 5~
$ fat.android <dbl> 1880, 963, 2460, 1203, 1933, 527, 1663, 3183, 1240, 1~
$ fat.gynoid <dbl> 5064, 5032, 4779, 3739, 4087, 2740, 5217, 6278, 2309,~
$ fat.whole <dbl> 24467, 19708, 25374, 15044, 20278, 9869, 21542, 32375~
$ lean.left_arm <dbl> 1987, 1931, 2884, 1753, 2652, 2425, 1913, 2266, 3066,~
$ lean.left_leg <dbl> 7059, 7190, 10281, 6014, 8242, 7903, 6829, 8889, 9664~
$ lean.left_body <dbl> 9516, 10693, 13847, 9736, 11387, 10573, 8954, 11482, ~
$ lean.left_whole <dbl> 20305, 21778, 29332, 19143, 24185, 22946, 18809, 2431~
$ lean.right_arm <dbl> 2049, 2081, 2888, 1754, 2487, 2439, 1930, 2236, 3253,~
$ lean.right_leg <dbl> 7104, 7506, 10200, 6009, 8685, 7841, 6950, 8923, 9198~
$ lean.right_body <dbl> 9199, 10304, 14593, 9636, 10779, 9733, 8602, 10672, 1~
$ lean.right_whole <dbl> 19605, 21310, 29643, 18792, 23653, 21837, 19407, 2372~
$ lean.arms <dbl> 4036, 4012, 5773, 3508, 5139, 4864, 3843, 4501, 6319,~
$ lean.legs <dbl> 14163, 14696, 20482, 12023, 16928, 15744, 13779, 1781~
$ lean.body <dbl> 18715, 20998, 28440, 19372, 22166, 20306, 17556, 2215~
$ lean.android <dbl> 2669, 2782, 3810, 2455, 2904, 2656, 2297, 3094, 3344,~
$ lean.gynoid <dbl> 6219, 7209, 10233, 5866, 7525, 5970, 5825, 8175, 7760~
$ lean.whole <dbl> 39910, 43088, 58976, 37934, 47837, 44783, 38216, 4804~
$ BMC.left_arm <dbl> 181, 138, 204, 144, 180, 173, 140, 173, 220, 226, 225~
$ BMC.left_leg <dbl> 567, 508, 728, 441, 562, 574, 482, 631, 633, 630, 672~
$ BMC.left_body <dbl> 622, 414, 696, 367, 526, 465, 370, 629, 473, 629, 509~
$ BMC.left_whole <dbl> 1680, 1321, 1945, 1201, 1527, 1580, 1131, 1688, 1544,~
$ BMC.right_arm <dbl> 198, 150, 210, 142, 176, 183, 140, 176, 224, 251, 226~
$ BMC.right_leg <dbl> 574, 514, 739, 431, 552, 565, 491, 641, 622, 636, 690~
$ BMC.right_body <dbl> 592, 428, 730, 351, 502, 409, 358, 582, 420, 616, 483~
$ BMC.right_whole <dbl> 1582, 1288, 1958, 1130, 1451, 1466, 1229, 1668, 1478,~
$ BMC.arms <dbl> 379, 288, 414, 285, 356, 357, 280, 348, 444, 478, 451~
$ BMC.legs <dbl> 1142, 1022, 1467, 872, 1115, 1139, 974, 1272, 1255, 1~
$ BMC.body <dbl> 1214, 842, 1426, 718, 1028, 874, 728, 1211, 893, 1245~
$ BMC.android <dbl> 80, 57, 90, 44, 56, 54, 43, 77, 52, 72, 59, 60, 65, 5~
$ BMC.gynoid <dbl> 314, 285, 427, 245, 299, 262, 241, 379, 335, 378, 332~
$ BMC.whole <dbl> 3261, 2609, 3903, 2331, 2978, 3046, 2360, 3356, 3022,~

```

The data set contains 80 rows and 59 columns. The variables in the data set are described as part of the `exscidata` package and can be seen by typing `?dxadata` in the console.

We will work further with lean body mass data, these are

variables starting with `lean..` In addition we need variables describing observations like `participant`, `time`, `multiple`, `single`, `sex`, `include`, `height` and `weight`. To select these variables we can try a couple of different approaches. The `select` function can select variables by name. This means that we can simply list them:

```
exscidata::dxadata |> ①  
  select(participant, time, multiple, single) |> ②  
  print() ③
```

- ① Retrieve the data from the `exscidata` package
- ② select variables based on names
- ③ Print the resulting data frame.

```
# A tibble: 80 x 4  
  participant time multiple single  
  <chr>      <chr> <chr>    <chr>  
1 FP28      pre    L        R  
2 FP40      pre    R        L  
3 FP21      pre    R        L  
4 FP34      pre    R        L  
5 FP23      pre    R        L  
6 FP26      pre    R        L  
7 FP36      pre    L        R  
8 FP38      pre    R        L  
9 FP25      pre    R        L  
10 FP19     pre    L        R  
# i 70 more rows
```

The above approach means a lot of work writing all columns names in the `select` call. An alternative approach is to select variables based on the first and last variable in a sequence. This is possible by using the syntax `<first column>:<last column>`.

```
exscidata::dxadata |>  
  select(participant:weight) |> ①  
  print()
```

- ① Selecting by the first and last column in a sequence.

```
# A tibble: 80 x 9
  participant time multiple single sex include age height weight
  <chr>      <chr> <chr>    <chr> <chr> <chr> <dbl> <dbl> <dbl>
1 FP28      pre  L       R     female incl  24.5  170  66.5
2 FP40      pre  R       L     female incl  22.1  175  64
3 FP21      pre  R       L     male   incl  26.8  184  85
4 FP34      pre  R       L     female incl  23.1  164  53
5 FP23      pre  R       L     male   incl  24.8  176.  68.5
6 FP26      pre  R       L     female excl  24.2  163  56
7 FP36      pre  L       R     female incl  20.5  158  60.5
8 FP38      pre  R       L     female incl  20.6  181  83.5
9 FP25      pre  R       L     male   incl  37.4  183  65
10 FP19     pre  L       R     male   incl  22.3  178.  73.5
# i 70 more rows
```

We also like to have the lean body mass data included in our new data set. Since all variables containing lean body mass data starts with `lean.` we can use a helper function to select them. Two alternatives are possible:

```
exscidata::dxadata |>
  select(participant:weight, starts_with("lean.")) |> ①
  print()

exscidata::dxadata |>
  select(participant:weight, contains("lean.")) |> ②
  print()
```

- ① Using `starts_with` to select all columns that starts with `lean.`
- ② Using `contains` to select all variables that contains `lean.`

```
# A tibble: 80 x 23
  participant time multiple single sex include age height weight
  <chr>      <chr> <chr>    <chr> <chr> <chr> <dbl> <dbl> <dbl>
1 FP28      pre  L       R     female incl  24.5  170  66.5
2 FP40      pre  R       L     female incl  22.1  175  64
```

```

3 FP21      pre  R      L      male  incl    26.8  184    85
4 FP34      pre  R      L      female incl    23.1  164    53
5 FP23      pre  R      L      male   incl    24.8  176.   68.5
6 FP26      pre  R      L      female excl    24.2  163    56
7 FP36      pre  L      R      female incl    20.5  158    60.5
8 FP38      pre  R      L      female incl    20.6  181    83.5
9 FP25      pre  R      L      male   incl    37.4  183    65
10 FP19     pre  L      R      male   incl    22.3  178.   73.5
# i 70 more rows
# i 14 more variables: lean.left_arm <dbl>, lean.left_leg <dbl>,
#   lean.left_body <dbl>, lean.left_whole <dbl>, lean.right_arm <dbl>,
#   lean.right_leg <dbl>, lean.right_body <dbl>, lean.right_whole <dbl>,
#   lean.arms <dbl>, lean.legs <dbl>, lean.body <dbl>, lean.android <dbl>,
#   lean.gynoid <dbl>, lean.whole <dbl>
# A tibble: 80 x 23
  participant time multiple single sex include age height weight
  <chr>      <chr> <chr>    <chr> <chr> <chr> <dbl> <dbl> <dbl>
1 FP28      pre  L      R      female incl    24.5  170    66.5
2 FP40      pre  R      L      female incl    22.1  175    64
3 FP21      pre  R      L      male   incl    26.8  184    85
4 FP34      pre  R      L      female incl    23.1  164    53
5 FP23      pre  R      L      male   incl    24.8  176.   68.5
6 FP26      pre  R      L      female excl    24.2  163    56
7 FP36      pre  L      R      female incl    20.5  158    60.5
8 FP38      pre  R      L      female incl    20.6  181    83.5
9 FP25      pre  R      L      male   incl    37.4  183    65
10 FP19     pre  L      R      male   incl    22.3  178.   73.5
# i 70 more rows
# i 14 more variables: lean.left_arm <dbl>, lean.left_leg <dbl>,
#   lean.left_body <dbl>, lean.left_whole <dbl>, lean.right_arm <dbl>,
#   lean.right_leg <dbl>, lean.right_body <dbl>, lean.right_whole <dbl>,
#   lean.arms <dbl>, lean.legs <dbl>, lean.body <dbl>, lean.android <dbl>,
#   lean.gynoid <dbl>, lean.whole <dbl>

```

There are other select helper functions such as `ends_with` and `matches` that work in a similar fashion as the above. `all_of` and `any_of` helps you select variables based on a vector of variables, `where` selects based on where a function of your choosing returns true. See `?select` for a complete list.

In a `select` call we can also rename variables using the syntax

<new name> = <old name>. Let's say we want to select and rename participant:

```
exscidata::dxadata |>
  select(parti = participant, time:weight, starts_with("lean. ")) |>
  print()
```

```
# A tibble: 80 x 23
  parti time multiple single sex include age height weight lean.left_arm
  <chr> <chr> <chr> <chr> <chr> <chr> <dbl> <dbl> <dbl> <dbl>
1 FP28 pre L R female incl 24.5 170 66.5 1987
2 FP40 pre R L female incl 22.1 175 64 1931
3 FP21 pre R L male incl 26.8 184 85 2884
4 FP34 pre R L female incl 23.1 164 53 1753
5 FP23 pre R L male incl 24.8 176. 68.5 2652
6 FP26 pre R L female excl 24.2 163 56 2425
7 FP36 pre L R female incl 20.5 158 60.5 1913
8 FP38 pre R L female incl 20.6 181 83.5 2266
9 FP25 pre R L male incl 37.4 183 65 3066
10 FP19 pre L R male incl 22.3 178. 73.5 3760
# i 70 more rows
# i 13 more variables: lean.left_leg <dbl>, lean.left_body <dbl>,
# lean.left_whole <dbl>, lean.right_arm <dbl>, lean.right_leg <dbl>,
# lean.right_body <dbl>, lean.right_whole <dbl>, lean.arms <dbl>,
# lean.legs <dbl>, lean.body <dbl>, lean.android <dbl>, lean.gynoid <dbl>,
# lean.whole <dbl>
```

Notice how different ways of selecting variables can be combined in `select`.

The `rename` function makes it easy to rename variables without the need to select.

```
exscidata::dxadata |>
  select(participant:weight, starts_with("lean. ")) |>
  rename(parti = participant) |>
  print()
```

```
# A tibble: 80 x 23
  parti time multiple single sex include age height weight lean.left_arm
```

	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<dbl>	<dbl>	<dbl>	<dbl>
1	FP28	pre	L	R	female	incl	24.5	170	66.5	1987
2	FP40	pre	R	L	female	incl	22.1	175	64	1931
3	FP21	pre	R	L	male	incl	26.8	184	85	2884
4	FP34	pre	R	L	female	incl	23.1	164	53	1753
5	FP23	pre	R	L	male	incl	24.8	176.	68.5	2652
6	FP26	pre	R	L	female	excl	24.2	163	56	2425
7	FP36	pre	L	R	female	incl	20.5	158	60.5	1913
8	FP38	pre	R	L	female	incl	20.6	181	83.5	2266
9	FP25	pre	R	L	male	incl	37.4	183	65	3066
10	FP19	pre	L	R	male	incl	22.3	178.	73.5	3760

```
# i 70 more rows
# i 13 more variables: lean.left_leg <dbl>, lean.left_body <dbl>,
#   lean.left_whole <dbl>, lean.right_arm <dbl>, lean.right_leg <dbl>,
#   lean.right_body <dbl>, lean.right_whole <dbl>, lean.arms <dbl>,
#   lean.legs <dbl>, lean.body <dbl>, lean.android <dbl>, lean.gynoid <dbl>,
#   lean.whole <dbl>
```

If we want to change the order of variables in a data set we can specify the order in a select call, or use `relocate`

```
exscidata::dxadata |>
  select(participant:weight, starts_with("lean.")) |>
  relocate(lean.whole) |>
  print()
```

```
# A tibble: 80 x 23
  lean.whole participant time multiple single sex include age height
      <dbl> <chr>      <chr> <chr> <chr> <chr> <chr> <dbl> <dbl>
1    39910 FP28      pre  L     R     female incl  24.5  170
2    43088 FP40      pre  R     L     female incl  22.1  175
3    58976 FP21      pre  R     L     male   incl  26.8  184
4    37934 FP34      pre  R     L     female incl  23.1  164
5    47837 FP23      pre  R     L     male   incl  24.8  176.
6    44783 FP26      pre  R     L     female excl  24.2  163
7    38216 FP36      pre  L     R     female incl  20.5  158
8    48045 FP38      pre  R     L     female incl  20.6  181
9     54710 FP25      pre  R     L     male   incl  37.4  183
10    58740 FP19      pre  L     R     male   incl  22.3  178.
# i 70 more rows
```

```
# i 14 more variables: weight <dbl>, lean.left_arm <dbl>, lean.left_leg <dbl>,
#   lean.left_body <dbl>, lean.left_whole <dbl>, lean.right_arm <dbl>,
#   lean.right_leg <dbl>, lean.right_body <dbl>, lean.right_whole <dbl>,
#   lean.arms <dbl>, lean.legs <dbl>, lean.body <dbl>, lean.android <dbl>,
#   lean.gynoid <dbl>
```

`relocate` puts the selected variable as the first column in the data set. If we want to specify the location we can use the arguments `.before` or `.after`.

```
exscidata::dxadata |>
  select(participant:weight, starts_with("lean.")) |>
  relocate(lean.whole, .after = sex) |>
  print()
```

```
# A tibble: 80 x 23
  participant time multiple single sex lean.whole include age height
  <chr>        <chr> <chr>    <chr> <chr>    <dbl> <chr>    <dbl> <dbl>
1 FP28      pre    L        R    female  39910 incl    24.5  170
2 FP40      pre    R        L    female  43088 incl    22.1  175
3 FP21      pre    R        L    male    58976 incl    26.8  184
4 FP34      pre    R        L    female  37934 incl    23.1  164
5 FP23      pre    R        L    male    47837 incl    24.8  176.
6 FP26      pre    R        L    female  44783 excl    24.2  163
7 FP36      pre    L        R    female  38216 incl    20.5  158
8 FP38      pre    R        L    female  48045 incl    20.6  181
9 FP25      pre    R        L    male    54710 incl    37.4  183
10 FP19     pre    L        R    male    58740 incl    22.3  178.
```

```
# i 70 more rows
```

```
# i 14 more variables: weight <dbl>, lean.left_arm <dbl>, lean.left_leg <dbl>,
#   lean.left_body <dbl>, lean.left_whole <dbl>, lean.right_arm <dbl>,
#   lean.right_leg <dbl>, lean.right_body <dbl>, lean.right_whole <dbl>,
#   lean.arms <dbl>, lean.legs <dbl>, lean.body <dbl>, lean.android <dbl>,
#   lean.gynoid <dbl>
```

2.0.2.2 Creating new variables

`mutate` let's us create new variables in a data set. These can be a function of variables already in the data set or created from our input.

Let's create a variable representing the percentage of lean mass to body mass.

```
exscidata::dxadata |>
  select(participant:weight, starts_with("lean.")) |>
  mutate(rel_lean_whole = 100 * ((lean.whole/1000) / weight)) |>
  relocate(rel_lean_whole) |>
  print()
```

A tibble: 80 x 24

	rel_lean_whole	participant	time	multiple	single	sex	include	age	height
	<dbl>	<chr>	<chr>	<chr>	<chr>	<chr>	<chr>	<dbl>	<dbl>
1	60.0	FP28	pre	L	R	female	incl	24.5	170
2	67.3	FP40	pre	R	L	female	incl	22.1	175
3	69.4	FP21	pre	R	L	male	incl	26.8	184
4	71.6	FP34	pre	R	L	female	incl	23.1	164
5	69.8	FP23	pre	R	L	male	incl	24.8	176.
6	80.0	FP26	pre	R	L	female	excl	24.2	163
7	63.2	FP36	pre	L	R	female	incl	20.5	158
8	57.5	FP38	pre	R	L	female	incl	20.6	181
9	84.2	FP25	pre	R	L	male	incl	37.4	183
10	79.9	FP19	pre	L	R	male	incl	22.3	178.

i 70 more rows

i 15 more variables: weight <dbl>, lean.left_arm <dbl>, lean.left_leg <dbl>,
 # lean.left_body <dbl>, lean.left_whole <dbl>, lean.right_arm <dbl>,
 # lean.right_leg <dbl>, lean.right_body <dbl>, lean.right_whole <dbl>,
 # lean.arms <dbl>, lean.legs <dbl>, lean.body <dbl>, lean.android <dbl>,
 # lean.gynoid <dbl>, lean.whole <dbl>

2.0.3 From wide to long and back again

The `dxadata` data set is not a tidy data set. It contains two variables (`single` and `multiple`) that indicates which leg has been training with low and moderate volume respectively (Hammarström et al. 2020). Additionally, lean mass variables could be separated based on body half (right or left). To compare training volume we need to reformat the data set. We will start by making a smaller data set that indicate training volume per leg.

As we have already seen, `participant`, `single` and `multiple` are the variables needed to make a data set that indicates training volume per leg, per participant. We will start by selecting these columns followed by pivoting the data as the volume data is located in two variables. This essentially means that we will make the data set longer.

```
exscidata::dxadata |>
  select(participant, multiple, single) |> ①
  pivot_longer(values_to = "leg", names_to = "volume", cols = multiple:single) %>% ②
  print()
```

- ① Selecting our variables of interest
- ② Creating a long data set based on volume data spread over two columns.

```
# A tibble: 160 x 3
  participant volume   leg
  <chr>      <chr>   <chr>
1 FP28      multiple L
2 FP28      single   R
3 FP40      multiple R
4 FP40      single   L
5 FP21      multiple R
6 FP21      single   L
7 FP34      multiple R
8 FP34      single   L
9 FP23      multiple R
10 FP23     single   L
# i 150 more rows
```

As we see we now have a long data set, but it is longer than expected. The original data contains only 41 participants. As each participant has two legs we would expect 82 observations. Above we did not remove post-intervention observations and we therefore have several duplicates. This can be taken care of by using the `distinct` function which returns unique observations across a combination of variables.

```

participant_volume <- exscidata::dxadata |>
  select(participant, multiple, single) |> #
  pivot_longer(values_to = "leg", names_to = "volume", cols = multiple:single) %>%
  distinct(participant, volume, leg) %>% ①
  print()

```

- ① Removes all duplicate combinations of participant, volume and leg.

```

# A tibble: 82 x 3
  participant volume leg
  <chr>      <chr> <chr>
1 FP28      multiple L
2 FP28      single  R
3 FP40      multiple R
4 FP40      single  L
5 FP21      multiple R
6 FP21      single  L
7 FP34      multiple R
8 FP34      single  L
9 FP23      multiple R
10 FP23     single  L
# i 72 more rows

```

We can save our smaller data set as `participant_volume`

Next we want to create a data set of right and left leg lean mass data. We will start by selecting variables.

```

exscidata::dxadata |>
  select(participant, time, starts_with("lean.") & ends_with("_leg")) |> #
  print()

```

```

# A tibble: 80 x 4
  participant time lean.left_leg lean.right_leg
  <chr>      <chr>      <dbl>      <dbl>
1 FP28      pre          7059          7104
2 FP40      pre          7190          7506
3 FP21      pre         10281         10200

```

```

4 FP34      pre      6014      6009
5 FP23      pre      8242      8685
6 FP26      pre      7903      7841
7 FP36      pre      6829      6950
8 FP38      pre      8889      8923
9 FP25      pre      9664      9198
10 FP19     pre      9704      9806
# i 70 more rows

```

Notice how `&` was used to create a conditional selection of variables. In addition to selecting time, include³ and participant we select variables that starts with `lean.` AND ends with `_leg`.

The resulting data set is wide. Two variables contains the same variable (lean mass), but one variable (leg) is lurking in two variables (`lean.left_leg` and `lean.right_leg`). Let's make the data set long.

```

leg_leanmass <- exscidata::dxadata |> ③
  select(participant, time, include, starts_with("lean.") & ends_with("_leg")) |>
  pivot_longer(names_to = "leg", values_to = "lean_mass", cols = contains("lean.)) |> ①
  mutate(leg = if_else(leg == "lean.left_leg", "L", "R")) |> ②

print()

```

- ① Notice how select helpers can be used in `pivot_longer`.
- ② We use `mutate` together with `if_else` to change the leg indicator to R and L
- ③ We save the data set as `leg_leanmass`

```

# A tibble: 160 x 5
  participant time  include leg  lean_mass
  <chr>      <chr> <chr>  <chr>    <dbl>
1 FP28      pre   incl   L        7059
2 FP28      pre   incl   R        7104
3 FP40      pre   incl   L        7190
4 FP40      pre   incl   R        7506

```

³The variable `incl` is used to indicate which participants to include in a final analysis. Included participants completed a given set of training sessions.

```

5 FP21      pre    incl    L        10281
6 FP21      pre    incl    R        10200
7 FP34      pre    incl    L         6014
8 FP34      pre    incl    R         6009
9 FP23      pre    incl    L         8242
10 FP23     pre    incl    R         8685
# i 150 more rows

```

`pivot_longer` has a brother called `pivot_wider`, this function performs the reverse operation making long data sets wide. Let's say that we would like to calculate the paired difference of leg lean mass from pre to post, we could make a wider data set and calculate `post - pre`

```

exscidata::dxadata |>
  select(participant, time, include, starts_with("lean.") & ends_with("_leg")) |>
  pivot_longer(names_to = "leg", values_to = "lean_mass", cols = contains("lean.")) |>
  mutate(leg = if_else(leg == "lean.left_leg", "L", "R")) |>
  pivot_wider(names_from = time, values_from = lean_mass) |> ①
  mutate(delta_lean_mass = post - pre) |> ②
  print()

```

- ① Pivot wider creates new columns based on names in `time` and values in `lean_mass`
- ② The new variable is calculated as the difference between pre and post-intervention values.

```

# A tibble: 82 x 6
  participant include leg    pre  post delta_lean_mass
  <chr>         <chr> <chr> <dbl> <dbl>         <dbl>
1 FP28         incl    L     7059  7273           214
2 FP28         incl    R     7104  7227           123
3 FP40         incl    L     7190  7192             2
4 FP40         incl    R     7506  7437           -69
5 FP21         incl    L    10281 10470           189
6 FP21         incl    R    10200 10819           619
7 FP34         incl    L     6014  6326           312
8 FP34         incl    R     6009  6405           396
9 FP23         incl    L     8242  8687           445
10 FP23        incl    R     8685  8480          -205
# i 72 more rows

```

2.0.4 Joining data sets

We have constructed two smaller data sets, one indicating which leg performed low and moderate volume and one data set containing the lean mass values for each leg, pre- and post-intervention. Next step is to join the two.

`dplyr` contains functions for joining data frames. There are, as illustrated in Figure 2.1 important differences between the functions where outer joins (left, right and full) keeps all observations in `x`, `y` and both `x` and `y` respectively. `inner_join` however, drops unmatched observations from both input data frames. Unlike the others, `anti_join` function removes observations in `x` that is present in `y`.

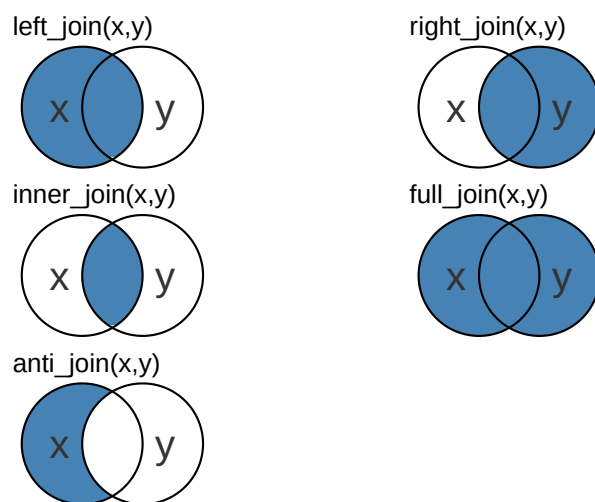


Figure 2.1: Functions for joining data sets.

Our two data sets are pretty insensitive to dropped observations since the two data sets should be complete. We will use a `left_join` to put the data set together. This will match participant and leg because these two variables exist in both data sets. If we want to join by other variables we may specify such a variable. We'll save the joined data set as `leg_leanmass`.

```
leg_leanmass <- leg_leanmass |>  
  inner_join(participant_volume) |>  
  print()
```

Joining with `by = join\_by(participant, leg)`

```
# A tibble: 160 x 6
  participant time include leg lean_mass volume
  <chr>      <chr> <chr>  <chr>    <dbl> <chr>
1 FP28      pre  incl   L        7059 multiple
2 FP28      pre  incl   R        7104 single
3 FP40      pre  incl   L        7190 single
4 FP40      pre  incl   R        7506 multiple
5 FP21      pre  incl   L       10281 single
6 FP21      pre  incl   R       10200 multiple
7 FP34      pre  incl   L        6014 single
8 FP34      pre  incl   R        6009 multiple
9 FP23      pre  incl   L        8242 single
10 FP23     pre  incl   R        8685 multiple
# i 150 more rows
```

2.0.5 Filters and sorting rows

We included the variable `include` in our data set, this will make it possible to get rid of observations from participants that should be part of the final analysis. We can use `filter` to perform this operation.

```
leg_leanmass |>
  filter(include == "incl") |>
  print()
```

```
# A tibble: 136 x 6
  participant time include leg lean_mass volume
  <chr>      <chr> <chr>  <chr>    <dbl> <chr>
1 FP28      pre  incl   L        7059 multiple
2 FP28      pre  incl   R        7104 single
3 FP40      pre  incl   L        7190 single
4 FP40      pre  incl   R        7506 multiple
5 FP21      pre  incl   L       10281 single
6 FP21      pre  incl   R       10200 multiple
7 FP34      pre  incl   L        6014 single
8 FP34      pre  incl   R        6009 multiple
```

```

 9 FP23      pre   incl   L           8242 single
10 FP23      pre   incl   R           8685 multiple
# i 126 more rows

```

The double equal sign can be read as “equal to” in R, a common source of confusion is that the single equal sign (=) is used as a assignment operator.⁴

The `filter` function keeps rows of a data frame where a condition that we specify returns `TRUE`. To construct testable conditions that allows for results being either `FALSE` or `TRUE` we will use one, or a combination operators, such as

<code>==</code>	→ “equal to”
<code>!=</code>	→ “not equal to”
<code>></code>	→ “larger than”
<code><</code>	→ “smaller than”
<code>>=, <=</code>	→ “larger/smaller or equal to”
<code>!</code>	→ “NOT”
<code>&</code>	→ “AND”
<code> </code>	→ “OR”

The above are all part of a collection of logical operators.⁵ We may demonstrate the mechanism by which `filter` operates by creating our own vector of `TRUE/FALSE` values. In the code chunk below a vector (`INCLUDE`) is created based on our `include` variable. A logical vector is created from the statement `INCLUDE != "excl"`. We then use this vector in the filter statement.

```

INCLUDE <- leg_leanmass |>
  pull(include)

INCLUDE <- INCLUDE != "excl"

INCLUDE

```

⁴An assignment operator is used to assign data values to objects stored in the work space. The commonly used `<-` is equivalent to `=`. Note however that `->` is different from `=`.

⁵See [here](#) for a technical overview of logical operators.

```

[1] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE FALSE FALSE
[13] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
[25] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
[37] TRUE TRUE TRUE TRUE FALSE FALSE TRUE TRUE TRUE TRUE TRUE FALSE FALSE
[49] FALSE FALSE FALSE FALSE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
[61] TRUE TRUE TRUE TRUE TRUE TRUE TRUE FALSE FALSE TRUE TRUE TRUE TRUE
[73] TRUE TRUE FALSE FALSE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
[85] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
[97] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
[109] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE FALSE FALSE TRUE TRUE
[121] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
[133] TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE TRUE
[145] TRUE TRUE FALSE FALSE TRUE TRUE FALSE FALSE TRUE TRUE TRUE TRUE
[157] FALSE FALSE FALSE FALSE

```

```

leg_leanmass |>
  filter(INCLUDE)

```

```

# A tibble: 136 x 6
  participant time include leg lean_mass volume
  <chr>        <chr> <chr>  <chr>    <dbl> <chr>
1 FP28      pre  incl   L      7059 multiple
2 FP28      pre  incl   R      7104 single
3 FP40      pre  incl   L      7190 single
4 FP40      pre  incl   R      7506 multiple
5 FP21      pre  incl   L     10281 single
6 FP21      pre  incl   R     10200 multiple
7 FP34      pre  incl   L      6014 single
8 FP34      pre  incl   R      6009 multiple
9 FP23      pre  incl   L      8242 single
10 FP23     pre  incl   R      8685 multiple
# i 126 more rows

```

In R, a vector of TRUE/FALSE (or in short form T/F) are is a special case which R will prohibit us from overwriting.

2.0.5.1 Grouped filters

Using the dplyr syntax it is easy to e.g. filter or mutate based on a grouping of the data set. The function `group_by` creates

a grouped data set. In the context of filtering in our data set we may want to filter out observations larger than the median from two time points.

```
leg_leanmass |>
  group_by(time) |>
  filter(lean_mass > median(lean_mass)) |>
  print()
```

```
# A tibble: 80 x 6
# Groups:   time [2]
  participant time include leg lean_mass volume
  <chr>      <chr> <chr>  <chr>    <dbl> <chr>
1 FP21      pre  incl   L      10281 single
2 FP21      pre  incl   R      10200 multiple
3 FP23      pre  incl   R       8685 multiple
4 FP38      pre  incl   L       8889 single
5 FP38      pre  incl   R       8923 multiple
6 FP25      pre  incl   L       9664 single
7 FP25      pre  incl   R       9198 multiple
8 FP19      pre  incl   L       9704 multiple
9 FP19      pre  incl   R       9806 single
10 FP13     pre  incl   L      10086 multiple
# i 70 more rows
```

The above operation removed exactly half of the observations, as expected since we wanted observations larger than the median from both time points. We may combine this statement with filtering away "excl" from the `include` variable.

```
leg_leanmass |>
  group_by(time) |>
  filter(include != "excl", lean_mass > median(lean_mass)) |>
  print()
```

```
# A tibble: 68 x 6
# Groups:   time [2]
  participant time include leg lean_mass volume
  <chr>      <chr> <chr>  <chr>    <dbl> <chr>
```

```

1 FP21      pre   incl   L           10281 single
2 FP21      pre   incl   R           10200 multiple
3 FP23      pre   incl   R            8685 multiple
4 FP38      pre   incl   L            8889 single
5 FP38      pre   incl   R            8923 multiple
6 FP25      pre   incl   L            9664 single
7 FP25      pre   incl   R            9198 multiple
8 FP19      pre   incl   L            9704 multiple
9 FP19      pre   incl   R            9806 single
10 FP13     pre   incl   L           10086 multiple
# i 58 more rows

```

Putting two conditions in a `filter` call is like explicitly using `&` (AND) to combine statements. All conditions must evaluate to be `TRUE` for the row to be included in final output.

An alternative to grouping by the `group_by` function is to do a “per-operation grouping”⁶ using the `.by` argument. This does not leave a grouping in the data frame after filtering.

```

leg_leanmass |>
  filter(include != "excl", lean_mass > median(lean_mass), .by = time) |>
  print()

```

```

# A tibble: 68 x 6
  participant time   include leg   lean_mass volume
  <chr>      <chr> <chr>   <chr>   <dbl> <chr>
1 FP21      pre   incl   L       10281 single
2 FP21      pre   incl   R       10200 multiple
3 FP23      pre   incl   R        8685 multiple
4 FP38      pre   incl   L        8889 single
5 FP38      pre   incl   R        8923 multiple
6 FP25      pre   incl   L        9664 single
7 FP25      pre   incl   R        9198 multiple
8 FP19      pre   incl   L        9704 multiple
9 FP19      pre   incl   R        9806 single
10 FP13     pre   incl   L       10086 multiple
# i 58 more rows

```

⁶See `?dplyr_by`

2.0.6 Summaries

We often want to reduce larger amount of data into some summaries, such as means, standard deviations, medians etc. These summaries may be calculated over any number of categorical variables representing e.g. groups of observations. Just like in the filtering statement above, we may work with a grouped data frame, or by using a per-operation grouping (`.by`).

We will calculate the median, first and third quartile, and minimum and maximum from each volume condition and time point. Below are both of the two alternatives of grouping used.

```
leg_leanmass |>
  filter(include != "excl") |>
  group_by(time, volume) |>
  summarise(Min = min(lean_mass),
            q25 = quantile(lean_mass, 0.25),
            Median = median(lean_mass),
            q75 = quantile(lean_mass, 0.75),
            Max = max(lean_mass)) |>
  print()
```

``summarise()`` has grouped output by 'time'. You can override using the ``groups`` argument.

```
# A tibble: 4 x 7
# Groups:   time [2]
  time volume    Min  q25 Median   q75   Max
  <chr> <chr>   <dbl> <dbl> <dbl> <dbl> <dbl>
1 post multiple 5343 7309. 8654. 10820. 13948
2 post single  5561 7201. 8772. 10453. 13526
3 pre  multiple 5500 7010. 8590. 10104. 13166
4 pre  single  5289 7126. 8654  10125. 13166
```

```
leanmass_sum <- leg_leanmass |>
  filter(include != "excl") |>
  summarise(Min = min(lean_mass),
            q25 = quantile(lean_mass, 0.25),
            Median = median(lean_mass),
```

```

      q75 = quantile(lean_mass, 0.75),
      Max = max(lean_mass),
      .by = c(time, volume)) |>
print()

```

```

# A tibble: 4 x 7
  time volume      Min  q25 Median   q75   Max
  <chr> <chr>    <dbl> <dbl> <dbl> <dbl> <dbl>
1 pre  multiple  5500 7010.  8590. 10104 13166
2 pre  single    5289 7126.  8654 10125. 13166
3 post multiple  5343 7309.  8654. 10820. 13948
4 post single    5561 7201.  8772. 10453. 13526

```

Notice how the data frames either have a persistent grouping, or no grouping depending on how we invoked grouping. Notice also that all values are nicely displayed, this is not always the case with summarise.

Consider the following example

```

vals <- c(4.5, 6.7, 4.6, 5.1, NA)

mean(vals)

```

```
[1] NA
```

The mean of a vector of values containing a missing value returns NA. If we where to have missing values (NA) in our original data we would have recieved a NA in results. To drop an NA from such a summary we need to specify `na.rm = TRUE` as part of a summary function.

```

vals <- c(4.5, 6.7, 4.6, 5.1, NA)

mean(vals, na.rm = TRUE)

```

```
[1] 5.225
```

R does not drop NA silently! This is a good thing as we want to get a notice when we are missing data. If we know we are missing data we need to explicitly type this in our calls. This “rule” is not always true as some functions silently drops NA, be aware!

2.0.7 Arranging data frames

At last we might want to arrange a data frame for ease of use, or prior to making a table etc. Arranging does not change the data frame *per se*, only how it is displayed.

We’ll used our summarized data frame from above and sort based on the `volume` variable.

```
leanmass_sum |>
  arrange(volume) |>
  print()
```

```
# A tibble: 4 x 7
  time volume      Min  q25 Median    q75   Max
  <chr> <chr>    <dbl> <dbl> <dbl> <dbl> <dbl>
1 pre  multiple  5500 7010.  8590. 10104 13166
2 post multiple  5343 7309.  8654. 10820. 13948
3 pre  single   5289 7126.  8654  10125. 13166
4 post single   5561 7201.  8772. 10453. 13526
```

Any other column will also work for arranging

```
leanmass_sum |>
  arrange(Median) |>
  print()
```

```
# A tibble: 4 x 7
  time volume      Min  q25 Median    q75   Max
  <chr> <chr>    <dbl> <dbl> <dbl> <dbl> <dbl>
1 pre  multiple  5500 7010.  8590. 10104 13166
2 post multiple  5343 7309.  8654. 10820. 13948
3 pre  single   5289 7126.  8654  10125. 13166
4 post single   5561 7201.  8772. 10453. 13526
```

Using a helper function (`desc`) reverses the sorting

```
leanmass_sum |>
  arrange(desc(Median)) |>
  print()
```

```
# A tibble: 4 x 7
  time volume      Min  q25 Median    q75   Max
  <chr> <chr>    <dbl> <dbl> <dbl> <dbl> <dbl>
1 post  single    5561 7201.  8772. 10453. 13526
2 pre   single    5289 7126.  8654  10125. 13166
3 post  multiple  5343 7309.  8654. 10820. 13948
4 pre   multiple  5500 7010.  8590. 10104  13166
```

Notice that `desc` also works for character data

```
leanmass_sum |>
  arrange(desc(volume)) |>
  print()
```

```
# A tibble: 4 x 7
  time volume      Min  q25 Median    q75   Max
  <chr> <chr>    <dbl> <dbl> <dbl> <dbl> <dbl>
1 pre   single    5289 7126.  8654  10125. 13166
2 post  single    5561 7201.  8772. 10453. 13526
3 pre   multiple  5500 7010.  8590. 10104  13166
4 post  multiple  5343 7309.  8654. 10820. 13948
```

->

3 More advanced features of ggplot2

As hinted above, `ggplot2` presents us with almost endless opportunities for graphical representations of our data. We will use the data we have wrangled above to better understand the `ggplot2` syntax.

In our summary of the lean mass data we have calculated a number of summary statistics that can be used to create a type of range-quartile-median plot not unlike the plots suggested by Tukey¹. We will use time as a categorical variable on the x-axis and our summary statistics displayed on the y-axis.

```
## Load participant data
participant_volume <- exscidata::dxadata |>
  select(participant, multiple, single) |>
  pivot_longer(values_to = "leg",
               names_to = "volume",
               cols = multiple:single) |>
  distinct(participant, volume, leg)

leg_leanmass <- exscidata::dxadata |>
  select(participant, time, include,
         starts_with("lean.") & ends_with("_leg")
        ) |>
  pivot_longer(names_to = "leg",
               values_to = "lean_mass",
               cols = contains("lean. ")) |>
  mutate(leg = if_else(leg == "lean.left_leg", "L", "R"))
```

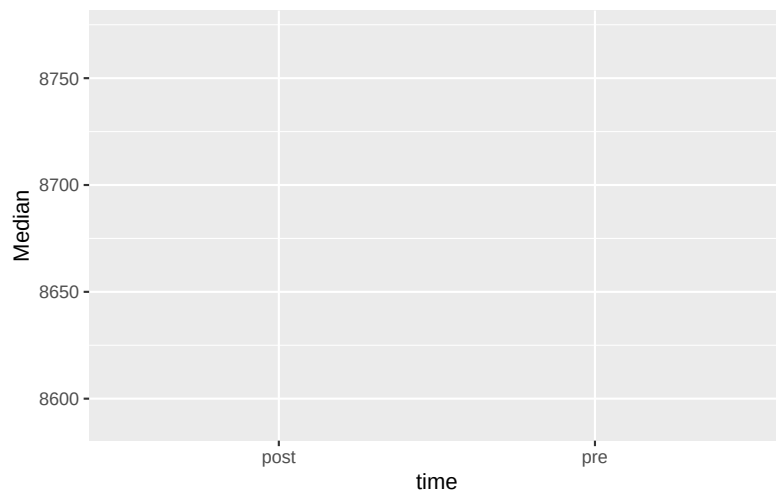
¹Tukey (1977) is considered to be creator of the box-plot. However, Spear may have suggested a predecessor to the box-plot in Charting statistics (Spear 1952).

```
leg_leanmass <- leg_leanmass |>
  inner_join(participant_volume)
```

Joining with `by = join\_by(participant, leg)`

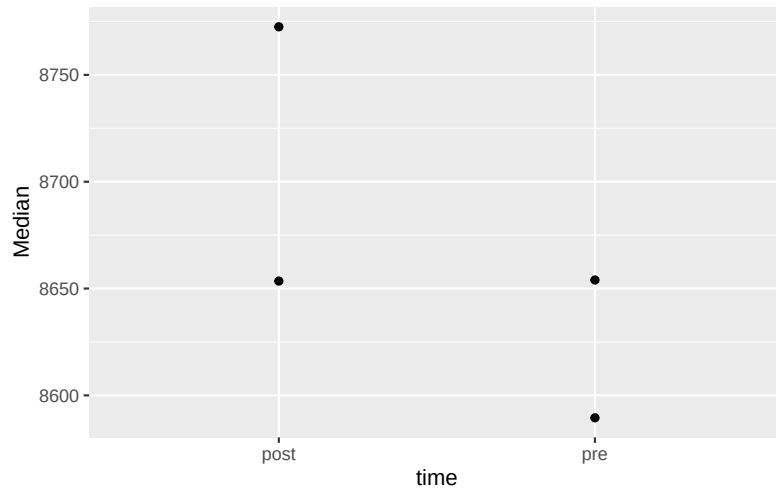
```
leanmass_sum <- leg_leanmass |>
  filter(include != "excl") |>
  summarise(Min = min(lean_mass),
            q25 = quantile(lean_mass, 0.25),
            Median = median(lean_mass),
            q75 = quantile(lean_mass, 0.75),
            Max = max(lean_mass),
            .by = c(time, volume))

ggplot(data = leanmass_sum, aes(time, Median))
```



We have specified a data set, and variables for the x and y axis. This specification gives us an empty coordinate system. We need to add geometric objects to represent the data mapped to x and y coordinates. Let's start simple


```
ggplot(data = leanmass_sum, aes(time, Median)) +
  geom_point()
```



The points now represents our median values per time point. There are a number of geometric objects, or geoms in **ggplot2**. You are also able to extend **ggplot2** by creating your own geoms, this has been done in many packages to make a specific geometric available. In **ggplot2** there are about 50 geoms available and in addition, a number of statistical transformations².

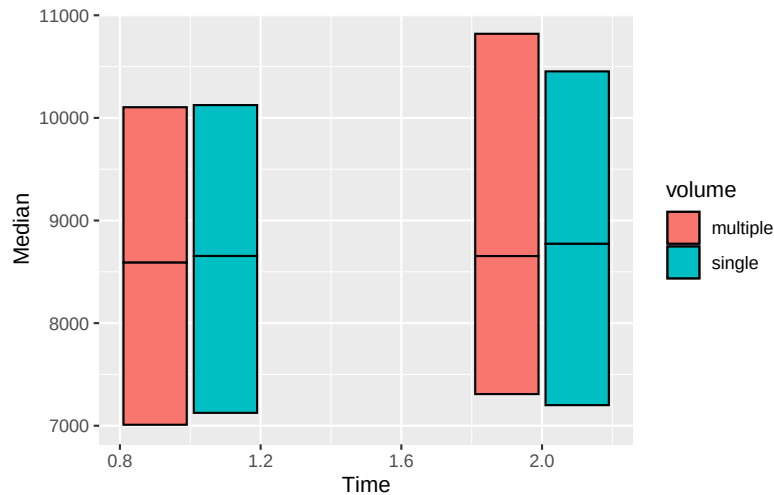
In our plot in the making, instead of points we may add rectangular shapes with a bit of reorganization of the data.

```
leanmass_sum |>
  mutate(Time = if_else(time == "pre", 1, 2)) |>
  ggplot(aes(Time, Median, fill = volume)) +

  geom_rect(aes(xmin = Time - 0.18,
                xmax = Time + 0.18,
                ymin = Median,
                ymax = q75),
            color = "black",
            position = position_dodge(width = 0.4)) +
```

²See <https://ggplot2.tidyverse.org/reference/> for a complete inventory.

```
geom_rect(aes(xmin = Time - 0.18,
              xmax = Time + 0.18,
              ymin = q25,
              ymax = Median),
          color = "black",
          position = position_dodge(width = 0.4))
```



We have hacked our way to a plot showing the median as a line inside a box covering the interquartile range. Let's add bars representing the minimum and maximum values. Note that we are using layers and we want our range bars to not over plot the boxes. We must therefore add them prior to the boxes. We will also scale the x-axis and add labels.

```
leanmass_sum |>
  mutate(Time = if_else(time == "pre", 1, 2)) |>
  ggplot(aes(Time, Median, fill = volume)) +

  geom_errorbar(aes(ymin = Min, ymax = Max),
                position = position_dodge(width = 0.4),
                width = 0) +

  geom_rect(aes(xmin = Time - 0.18,
```

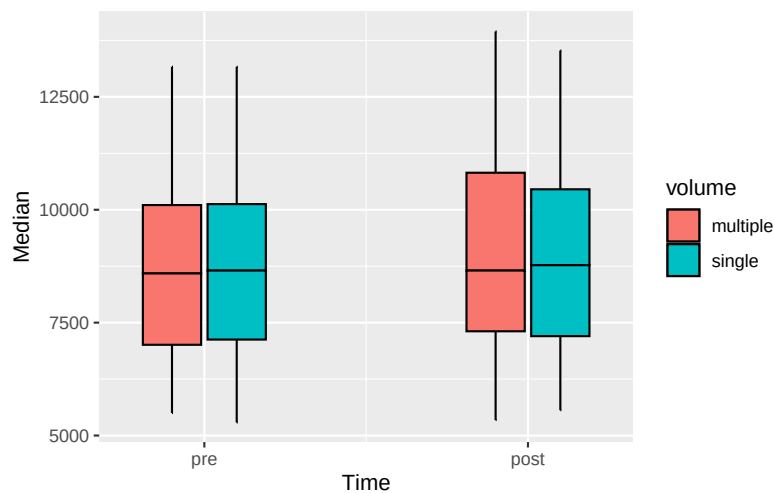
```

      xmax = Time + 0.18,
      ymin = Median,
      ymax = q75),
    color = "black",
    position = position_dodge(width = 0.4)) +

geom_rect(aes(xmin = Time - 0.18,
              xmax = Time + 0.18,
              ymin = q25,
              ymax = Median),
          color = "black",
          position = position_dodge(width = 0.4)) +

scale_x_continuous(limits = c(0.75, 2.25),
                   breaks = c(1, 2),
                   labels = c("pre", "post"))

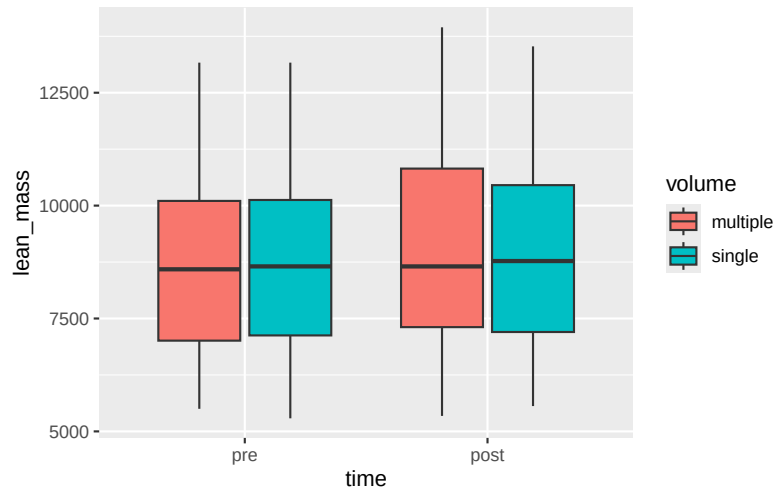
```



Do not bother creating the above figure! This is an example of the flexibility brought to us by using a set of simple geometric objects. There are shortcuts to the above plot that takes care of a couple of default operations. We need our data prior to being summarized with additional preparatory steps:

```
leg_leanmass |>
  filter(include != "excl") |>
  mutate(time = factor(time, levels = c("pre", "post"))) |>

ggplot(aes(time, lean_mass, fill = volume)) +
  geom_boxplot()
```



The new plot is a box-plot showing the same statistical transformations that we made by hand using a categorical x-axis with the time-values sorted as a factor.

```
a <- leanmass_sum |>
  mutate(Time = if_else(time == "pre", 1, 2)) |>
  ggplot(aes(Time, Median, fill = volume)) +

  geom_errorbar(aes(ymin = Min, ymax = Max),
    position = position_dodge(width = 0.4),
    width = 0) +

  geom_rect(aes(xmin = Time - 0.18,
    xmax = Time + 0.18,
    ymin = Median,
    ymax = q75),
```

```

        color = "black",
        position = position_dodge(width = 0.4)) +

geom_rect(aes(xmin = Time - 0.18,
              xmax = Time + 0.18,
              ymin = q25,
              ymax = Median),
          color = "black",
          position = position_dodge(width = 0.4)) +

scale_x_continuous(limits = c(0.75, 2.25),
                   breaks = c(1, 2),
                   labels = c("pre", "post")) +
theme(legend.position = "bottom")

b <- leg_leanmass |>
  filter(include != "excl") |>
  mutate(time = factor(time, levels = c("pre", "post"))) |>

ggplot(aes(time, lean_mass, fill = volume)) +
  geom_boxplot() +
  theme(legend.position = "bottom")

plot_grid(a, b, labels = c("a", "b"), ncol = 2)

```

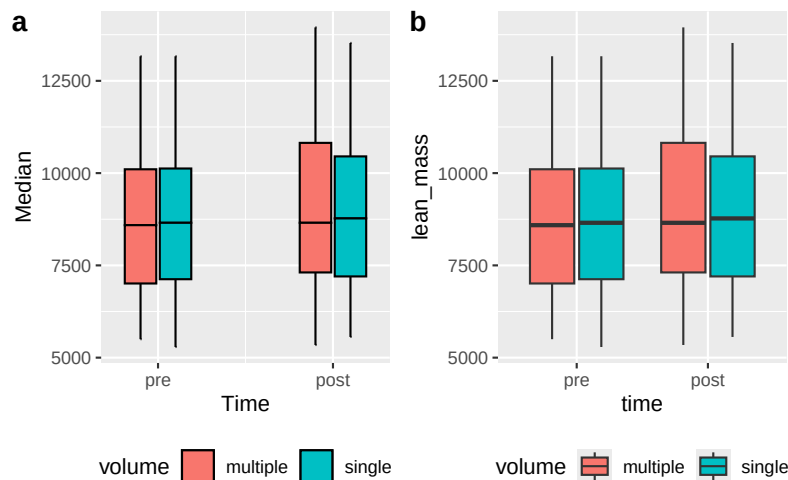


Figure 3.1: A direct comparison between (a) manual statistical summaries (transformations) and built in transformations in ggplot2 (b).

We have seen that `ggplot2` have some built in features that makes statistical transformation, but also that a similar result can be obtained by using other “geoms” such as `geom_rect`. Again, this allows for flexible creation of data visualizations.

3.0.1 Labels and themes

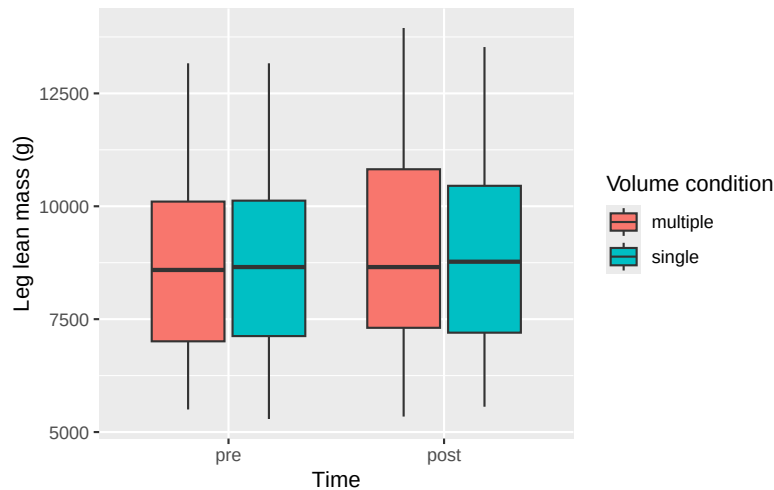
So far we have used the available data to determine what is being represented in the figure. To make the figure closer to “publication ready” we would want to take control over labels, colors and shapes that so far are presented with variable names and default settings.

The `labs` function adds customized labels to a `ggplot`. All aesthetic mappings are available for labeling. We will go further with the box-plot which uses `x`, `y` and `fill` as aesthetics mapped to data.

```
leg_leanmass |>
  filter(include != "excl") |>
  mutate(time = factor(time, levels = c("pre", "post"))) |>
```

```
ggplot(aes(time, lean_mass, fill = volume)) +
  geom_boxplot() +

  labs(x = "Time",
       y = "Leg lean mass (g)",
       fill = "Volume condition")
```



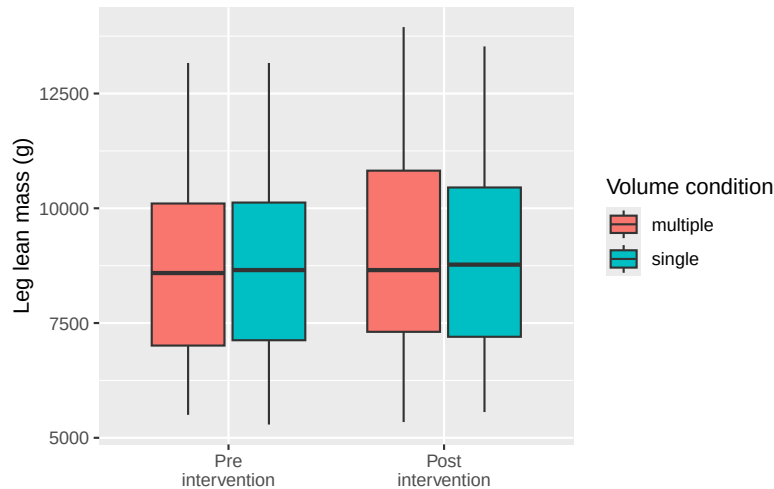
Additionally, we might want to think about if we need all axis labels. The time variable is specified with **pre** and **post** and won't need a overall axis label. Instead we might want to clean up the axis text. Since this is a data-driven element in our figure we can change it's behavior prior to plotting. We have already changed the order of the factor, let's add labels to each level and remove the overall **Time** label.

```
leg_leanmass |>
  filter(include != "excl") |>
  mutate(time = factor(time,
                        levels = c("pre", "post"),
                        labels = c("Pre\nintervention",
                                   "Post\nintervention"))) |>

  ggplot(aes(time, lean_mass, fill = volume)) +
    geom_boxplot() +
```

```
labs(x = "Time",
     y = "Leg lean mass (g)",
     fill = "Volume condition") +

theme(axis.title.x = element_blank())
```



Notice that we removed the “Time” label from the figure by removing it in the theme function (`theme(axis.title.x = element_blank())`; more about that later). The factor variable `time` was re-specified using the `factor` function. This function takes a character or factor variable and specifies levels (the order of factors) and optionally, labels for each level.

An alternative approach to changing the labels of the factor is to include labels in a call to `scale_x_discrete`.

```
leg_leanmass |>
  filter(include != "excl") |>
  mutate(time = factor(time,
                        levels = c("pre", "post"))) |>

ggplot(aes(time, lean_mass, fill = volume)) +
  geom_boxplot() +

  labs(x = "Time",
       y = "Leg lean mass (g)",
```



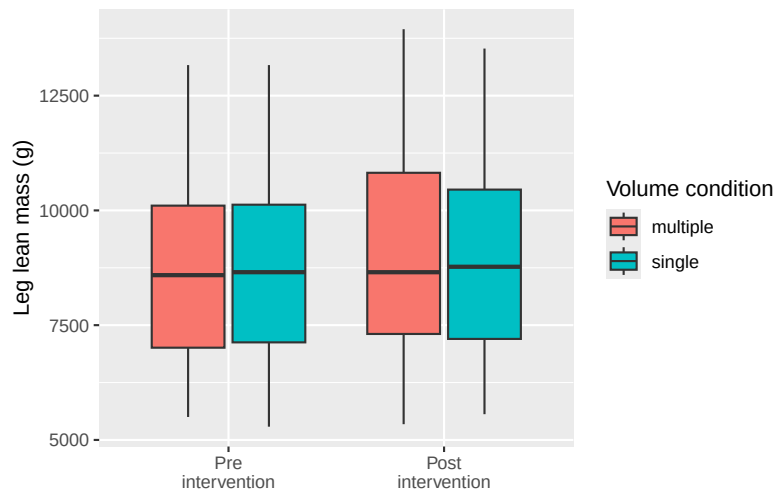
```

fill = "Volume condition") +

scale_x_discrete(labels = c("Pre\nintervention",
                             "Post\nintervention")) +

theme(axis.title.x = element_blank())

```



3.0.1.1 Theming

ggplot2 has a very flexible system for changing parts of a figure that are not directly connected to the data (i.e. created from aesthetic mapping). Each element in the theme that can be controlled is listed in the help pages for `theme`, type `?theme` in your console to access it.

The below illustrates some of the main components we might want to consider:

```

leg_leanmass |>
  filter(include != "excl") |>
  mutate(time = factor(time,
                        levels = c("pre", "post"))) |>

ggplot(aes(time, lean_mass, fill = volume)) +

```

```

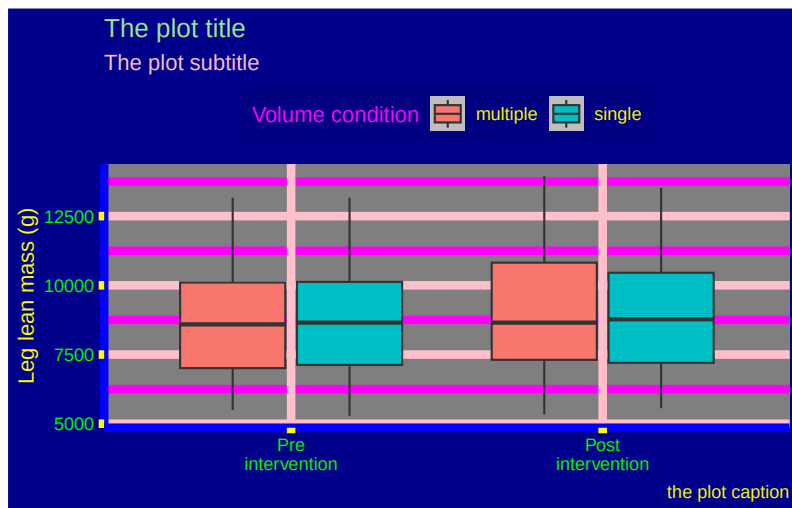
    geom_boxplot() +

labs(x = "Time",
     y = "Leg lean mass (g)",
     fill = "Volume condition",
     title = "The plot title",
     subtitle = "The plot subtitle",
     caption = "the plot caption") +

scale_x_discrete(labels = c("Pre\nintervention",
                           "Post\nintervention")) +

theme(axis.title.x = element_blank(),
      legend.position = "top",
      axis.title = element_text(color = "yellow"),
      axis.text = element_text(color = "green"),
      axis.ticks = element_line(color = "yellow", linewidth = 2),
      axis.line = element_line(color = "blue", linewidth = 2),
      panel.background = element_rect(fill = "gray50"),
      panel.grid.major = element_line(color = "pink", linewidth = 2),
      panel.grid.minor = element_line(color = "magenta", linewidth = 2),
      plot.background = element_rect(fill = "darkblue"),
      legend.background = element_rect(fill = "navyblue"),
      legend.key = element_rect(fill = "gray"),
      legend.text = element_text(color = "yellow"),
      legend.title = element_text(color = "magenta"),
      plot.caption = element_text(color = "yellow"),
      plot.title = element_text(color = "lightgreen"),
      plot.subtitle = element_text(color = "pink"))

```



Each element of the theme can be modified with an element function. The theme element `axis.title` is a text element and must subsequently be modified with `element_text()`. This function takes a number of arguments making it possible to modify text components. Below we use our basic plot and to modify the y axis title.

```
leg_leanmass |>
  filter(include != "excl") |>
  mutate(time = factor(time,
                        levels = c("pre", "post"))) |>

ggplot(aes(time, lean_mass, fill = volume)) +
  geom_boxplot() +

  labs(x = "Time",
       y = "Leg lean\nmass (g)",
       fill = "Volume condition",
       title = "The plot title",
       subtitle = "The plot subtitle",
       caption = "the plot caption") +

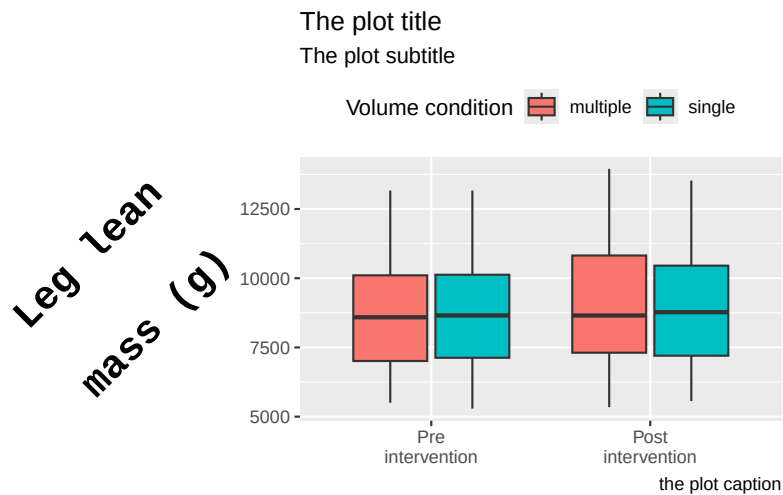
  scale_x_discrete(labels = c("Pre\nintervention",
                             "Post\nintervention")) +
```

```

theme(axis.title.x = element_blank(),
      legend.position = "top",
      axis.title.y = element_text(color = "black",
                                  family = "mono",
                                  face = "bold",

                                  size = 20,
                                  hjust = 0.5,
                                  vjust = 0.5,
                                  angle = 45,
                                  lineheight = 2),
      )

```



The element `axis.title.y` is similar to `axis.title.x` and if we want we could use `axis.title` to modify both for common attributes. Notice also that we have changed multiple numbers that have defaults. The `size` is the size of the text, `hjust` and `vjust` controls horizontal and vertical placement, respectively. `angle` rotates the text and `lineheight` controls the distance between lines (we have used `\n` to indicate a new line in the title).

Similarly to `element_text`, `element_rect` contains argument to control rectangular elements and `element_line` is used to control lines. `element_blank` is used to remove an element, we already used this above to remove the x axis title. In the same

“family” of function we find `margin` which can specify margins of theme elements.

In addition to controlling specific elements of a theme we have the option to use ready made themes. A couple of pre-specified themes are shipped with `ggplot2` (see Figure 3.2)

```
p <- leg_leanmass |>
  filter(include != "excl") |>
  mutate(time = factor(time,
                        levels = c("pre", "post"))) |>

ggplot(aes(time, lean_mass, fill = volume)) +
  geom_boxplot() +

  labs(x = "Time",
       y = "Leg lean\ncmass (g)",
       fill = "Volume condition",

       subtitle = "The plot subtitle",
       caption = "the plot caption") +

  scale_x_discrete(labels = c("Pre\nintervention",
                             "Post\nintervention"))

plot_grid(p + theme_bw() + labs(title = "theme_bw"),
          p + theme_classic() + labs(title = "theme_classic"),
          p + theme_minimal() + labs(title = "theme_minimal"),
          p + theme_dark() + labs(title = "theme_dark"),
          p + theme_light() + labs(title = "theme_light"),
          p + theme_linedraw() + labs(title = "theme_linedraw"),
          p + theme_bw() + labs(title = "theme_bw"),
          p + theme_grey() + labs(title = "theme_grey"),
          p + theme_void() + labs(title = "theme_void"),
          ncol = 2)
```

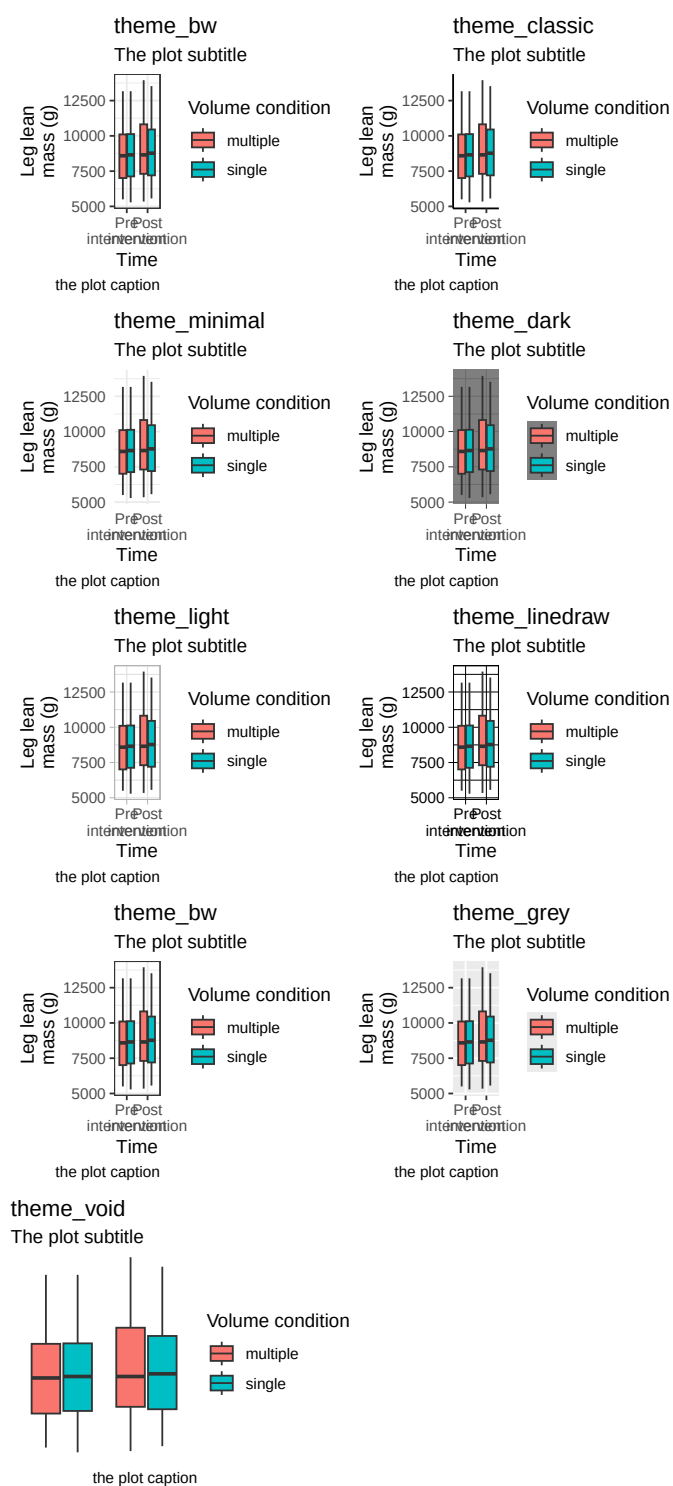


Figure 3.2: Examples of themes that are part of ggplot2

4 Scientific graphic design

An important rule of scientific graphics is, not to lie. Or as stated by Tufte (2001), “graphical excellence begins with telling the truth about the data”. Clear and honest representation of the data together with detailed labeling are key factors for avoiding lies (Tufte 2001).

Another key aspect in the construction of scientific graphs is to decide what to plot. Graphs are very good at showing differences, however, what differences should be show? In a two-group design (treatment vs. control) we should aim at showing the difference between groups, not the difference within groups (Ho et al. 2019). When data are grouped, or measurements repeated within experimental units (such as research participants), we should show aim at indicating this fact. When graphical summaries are shown, we should also aim at showing the underlying data (Tufte 2001).

4.0.1 Revise and edit

When trying to find a coherent guide for scientific graphics one quickly realize that guidelines are often in conflict with each other. We have to reach some pragmatic level of compromise in our work. How do we find this compromise? Tufte’s advice to constantly revise and edit is a good start (Tufte 2001). Like with writing we try to explain or show, but fail to do so. Multiple edits later our text is ready to communicate an idea to others, and we might have learned something too.

In practice a graph, or a collection of graphs are born from multiple revisions. Below we will revise our graph using a number of principles outlined by Tufte (Tufte 2001).

4.0.1.1 Show the data

Our original idea is shown below. We made a box-plot of raw lean mass values. A core shortcoming of this graph is that it hides uncertainty our sample size. We could add another layer on top of our bow-plots to show raw data. The geom `geom_point` is added with the additional arguments: `shape = 21`, `position = position_jitterdodge(dodge.width = 0.75, jitter.width = 0.1)`. The change in shape creates the possibility to fill points. As we have specified fill in aesthetics each point will have a condition-wise fill.

```
leg_leanmass |>
  filter(include != "excl") |>
  mutate(time = factor(time,
                        levels = c("pre", "post"))) |>

  ggplot(aes(time, lean_mass, fill = volume)) +
  geom_boxplot() +
  geom_point(shape = 21,
            position = position_jitterdodge(dodge.width = 0.75,
                                           jitter.width = 0.1))
```

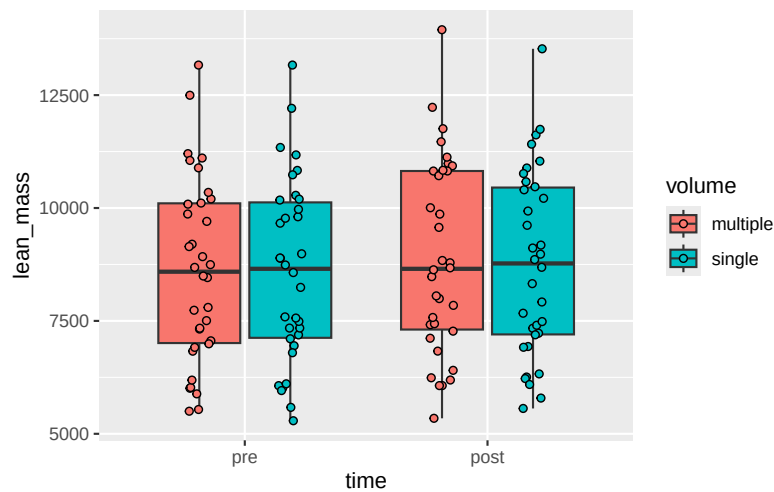


Figure 4.1: A boxplot with raw data plotted on top

Are we showing the data now? Some aspects of the data are still lost. The fact that the experiment is a nested one is not obvious from this display. Could we connect related data points?

In the next graph we'll try another alternative, using a line-graph we might be able to convey an aspect of the repeated measures design. We will also add a facet to show each volume condition.

```
leg_leanmass |>
  filter(include != "excl") |>
  mutate(time = factor(time,
                        levels = c("pre", "post"))) |>

  ggplot(aes(time, lean_mass, fill = volume, group = participant)) +

  geom_line() +
  geom_point(shape = 21) +
  facet_grid(. ~ volume)
```

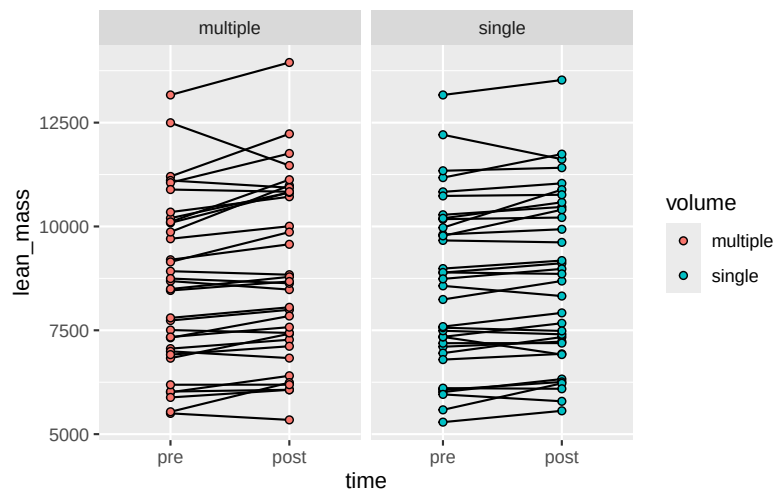


Figure 4.2: A line graph grouping observations per participant and volume condition

But what is the real comparison here? Here we might actually display within-condition differences. The analog to statistical

test might be that we show two within-condition t-tests and tell the reader that the effect in one group is greater than the other. This is not an honest test.

To revise further we might want to compare changes across conditions. We can do this and still highlight comparisons within participants since each participant performed both conditions. To accomplish this we need some data wrangling.

```
delta_lean <- leg_leanmass |>
  filter(include != "excl") |> ①
  pivot_wider(names_from = time, values_from = lean_mass) |> ②
  mutate(delta = post - pre) |> ③
  print()
```

- ① Using only included participants from our raw data set we first...
- ② create a wide data set by time allowing for...
- ③ a simple difference across time in each participant and condition.

```
# A tibble: 68 x 7
  participant include leg   volume    pre post delta
  <chr>         <chr> <chr> <chr>    <dbl> <dbl> <dbl>
1 FP28         incl  L     multiple 7059 7273 214
2 FP28         incl  R     single 7104 7227 123
3 FP40         incl  L     single 7190 7192 2
4 FP40         incl  R     multiple 7506 7437 -69
5 FP21         incl  L     single 10281 10470 189
6 FP21         incl  R     multiple 10200 10819 619
7 FP34         incl  L     single 6014 6326 312
8 FP34         incl  R     multiple 6009 6405 396
9 FP23         incl  L     single 8242 8687 445
10 FP23        incl  R     multiple 8685 8480 -205
# i 58 more rows
```

We have saved the data set in a new object called `delta_lean` we will use this for plotting.

```
delta_lean |>
  ggplot(aes(volume, delta, fill = volume, group = participant)) +
  geom_line() +
  geom_point(shape = 21)
```

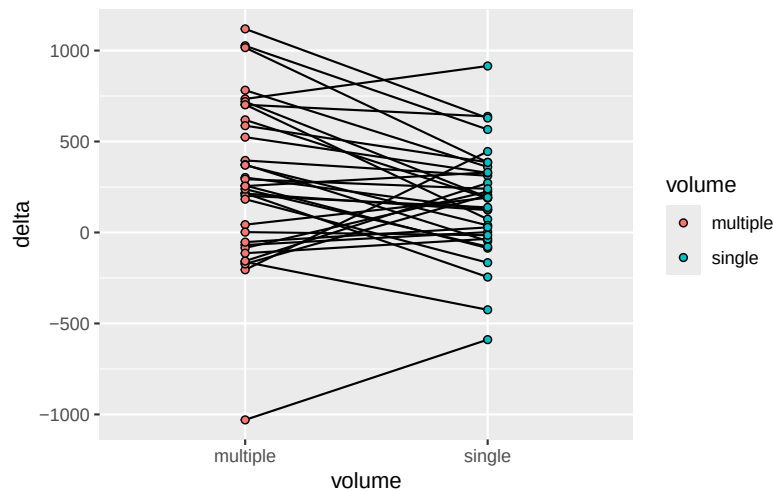


Figure 4.3: A line graph grouped by participant comparing conditions

It could be argued that the order of the factor variable `volume` is not accurately representing the data. Multiple is larger than single and the graph might be more accurate in this respect if we change this variable. I will do it as a pre-plotting operation. We may then also change the labels.

```
delta_lean |>
  mutate(volume = factor(volume, levels = c("single", "multiple"),
    labels = c("Low-volume", "Moderate-volume"))) |>
  ggplot(aes(volume, delta,
    fill = volume,
    group = participant)) +
  geom_line() +
  geom_point(shape = 21)
```

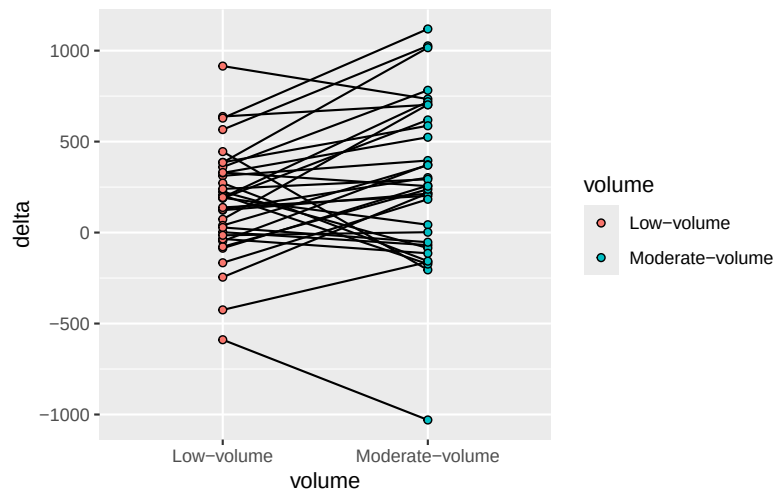


Figure 4.4: A line graph grouped by participant comparing conditions

4.0.1.2 Maximize the data-ink ratio

A guiding principle in Tufte's guideline is to remove non-data ink. We should aim to maximize the amount of ink used to display the data. Non-data ink can be removed by manipulating theme components. A great start might be to use a built in theme, `theme_minimal`.

```
delta_lean |>
  mutate(volume = factor(volume, levels = c("single", "multiple"),
                        labels = c("Low-volume", "Moderate-volume"))) |>

  ggplot(aes(volume, delta,
             fill = volume,
             group = participant)) +
  geom_line() +
  geom_point(shape = 21) +

  theme_minimal()
```

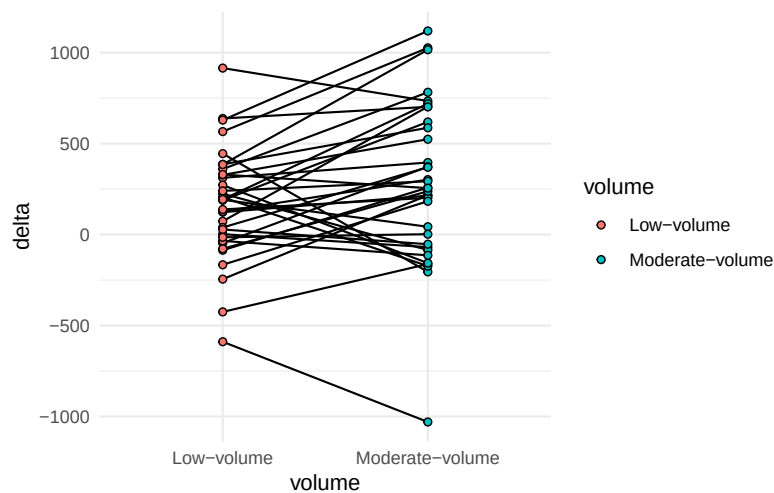


Figure 4.5: A line graph grouped by participant comparing conditions, reducing non-data ink with `theme_minimal`.

The legend is redundant, it can be removed since this information is already present in the x axis. The x-axis title is redundant as the information is duplicated in the labels. Some grid lines can be removed without loss of information, maybe even all of them. Let's add a line manually to show the 0 change. We can access these options using the `theme` function and elements manipulated therein. Placing the `theme` after `theme_minimal` in our call makes sure that we don't bring back elements from the pre-built theme. Let's also add a more descriptive label on the y-axis.

```
delta_lean |>
  mutate(volume = factor(volume, levels = c("single", "multiple"),
                          labels = c("Low-volume", "Moderate-volume"))) |>
  ggplot(aes(volume, delta,
             fill = volume,
             group = participant)) +
  geom_hline(yintercept = 0, color = "gray90") +
```

```

geom_line() +
geom_point(shape = 21) +

labs(y = "Change in lean mass from pre- to post-intervention (g)") +

theme_minimal() +

theme(legend.position = "none",
      axis.title.x = element_blank(),
      panel.grid.major.x = element_blank(),
      panel.grid.minor.y = element_blank(),
      panel.grid.major.y = element_blank())

```

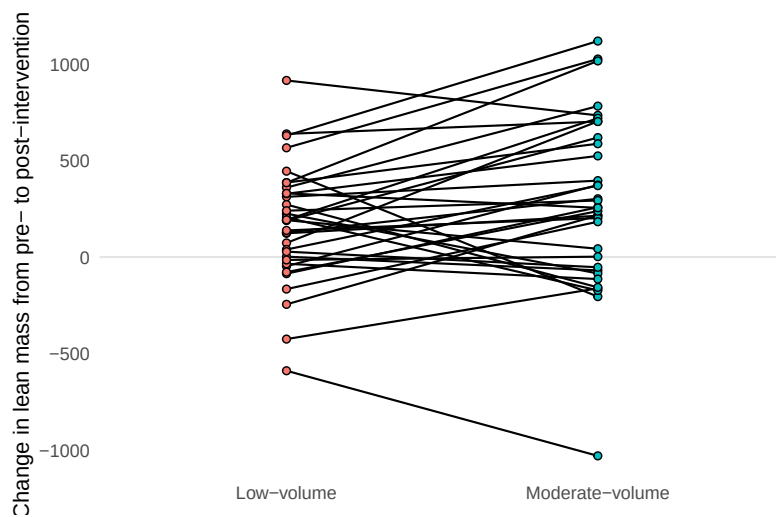


Figure 4.6: A line graph grouped by participant comparing conditions, reducing non-data ink with `theme_minimal`.

4.0.1.3 Erase redundant data ink

We have used points to represent the start and end of each line. This might be a case of redundant data ink.

```

delta_lean |>

  mutate(volume = factor(volume, levels = c("single", "multiple"),
                        labels = c("Low-volume", "Moderate-volume"))) |>

  ggplot(aes(volume, delta,
             fill = volume,
             group = participant)) +

  geom_hline(yintercept = 0, color = "gray90") +

  geom_line() +

  labs(y = "Change in lean mass from pre- to post-intervention (g)") +

  theme_minimal() +

  theme(legend.position = "none",
        axis.title.x = element_blank(),
        panel.grid.major.x = element_blank(),
        panel.grid.minor.y = element_blank(),
        panel.grid.major.y = element_blank())

```

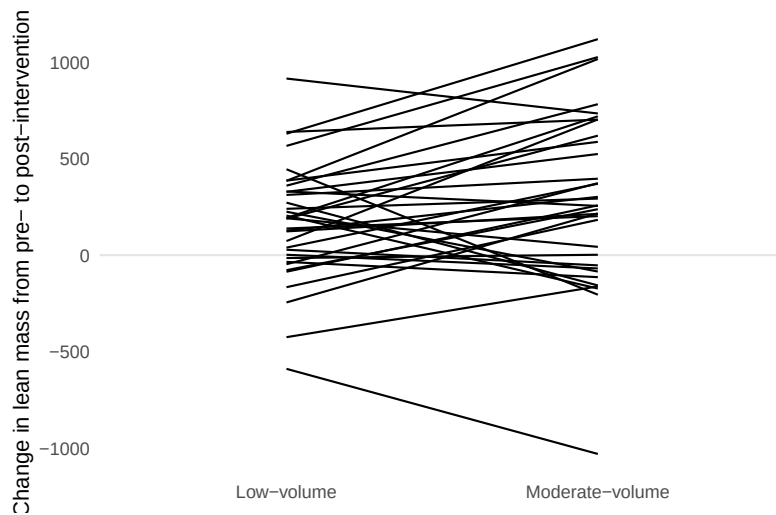


Figure 4.7: A line graph grouped by participant comparing conditions, reducing non-data ink with `theme_minimal` and removing redundant data ink by removing points.

To make the graph even more light weight we could consider making lines a bit transparent. Using `alpha = 0.5` we can add transparency to lines.

```
delta_lean |>
  mutate(volume = factor(volume, levels = c("single", "multiple"),
    labels = c("Low-volume", "Moderate-volume"))) |>
  ggplot(aes(volume, delta,
    fill = volume,
    group = participant)) +
  geom_hline(yintercept = 0, color = "gray90") +
  geom_line(alpha = 0.5) +
  labs(y = "Change in lean mass from pre- to post-intervention (g)") +
```



```
theme_minimal() +

theme(legend.position = "none",
      axis.title.x = element_blank(),
      panel.grid.major.x = element_blank(),
      panel.grid.minor.y = element_blank(),
      panel.grid.major.y = element_blank())
```

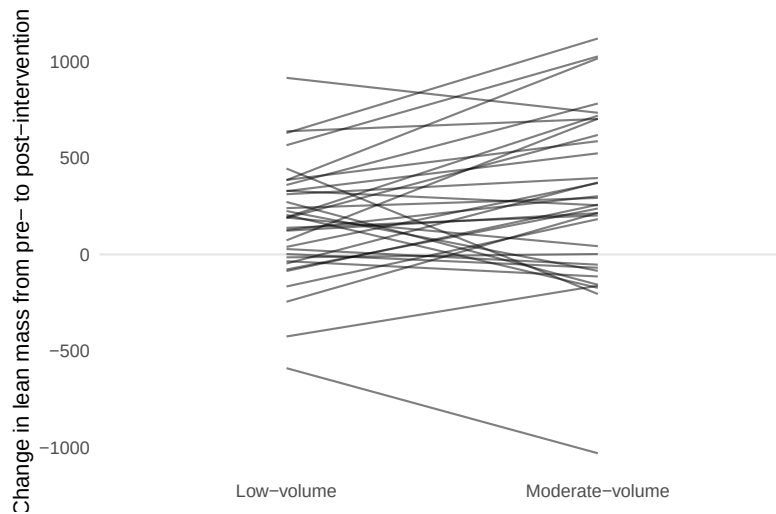


Figure 4.8: A line graph grouped by participant comparing conditions, reducing non-data ink with `theme_minimal` and removing redundant data ink by removing points and adding transparency.

4.0.1.4 Making comparisons

We now have a graph that represents the data, could we aid interpreting the figure? Perhaps we could add some information showing average changes per condition? Below we will construct a data frame of averages for each condition and add it to the figure.

```
descriptives <- delta_lean |>
  mutate(volume = factor(volume, levels = c("single", "multiple"),
                        labels = c("Low-volume", "Moderate-volume"))) |>
```

```

summarise(.by = volume,
          m = mean(delta))

delta_lean |>

mutate(volume = factor(volume, levels = c("single", "multiple"),
                      labels = c("Low-volume", "Moderate-volume"))) |>

ggplot(aes(volume, delta,
          fill = volume,
          group = participant)) +

geom_hline(yintercept = 0, color = "gray90") +

geom_point(data = filter(descriptives, volume== "Low-volume"), ①
          aes(volume, m, group = NULL, fill = NULL),
          position = position_nudge(x = -0.1)) +

  geom_point(data = filter(descriptives, volume== "Moderate-volume"), ②
          aes(volume, m, group = NULL, fill = NULL),
          position = position_nudge(x = 0.1)) +

geom_line(alpha = 0.5) +

labs(y = "Change in lean mass from pre- to post-intervention (g)") +

theme_minimal() +

theme(legend.position = "none",
      axis.title.x = element_blank(),
      panel.grid.major.x = element_blank(),
      panel.grid.minor.y = element_blank(),
      panel.grid.major.y = element_blank())

```

① Two point layers are added to make it easy to plot points be-

sides lines. Notice that the x-axis factor must be correctly formatted

- ② The other point layer, here `position_nudge` is positive.

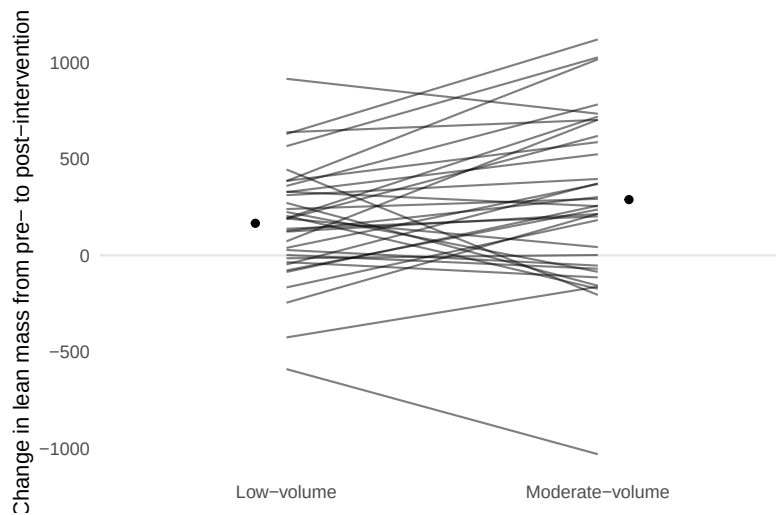


Figure 4.9: A line graph grouped by participant comparing conditions, reducing non-data ink with `theme_minimal` and removing redundant data ink by removing points and adding transparency. Adding group averages with `geom_point`.

Again, we might not be giving the reader of the graph the full story. Recently a new type of graph has been suggested to aid interpretation of two-group comparisons. In addition to means we could add the estimate of the difference between groups. These plots are called estimation plots (Ho et al. 2019). In our previous work we used different variations of ANCOVA models to adjust for baseline values (Hammarström et al. 2020) while modelling the change score. Let's start by fitting the model. Since the design is a bit tricky we need to let the model know that observations are grouped by participant. We can use the same `delta_lean` data set to fit the model.

```
library(lme4)

mod <- lmer(delta ~ scale(pre) + volume + (1|participant),
```

```
data = delta_lean)

mod_results <- data.frame(cbind(coef(summary(mod)), confint(mod)[3:5, ])) ①
```

- ① The output from the model is saved in a new object, including confidence intervals for each coefficient.

Our model gives us one coefficient of interest, `volumesingle`. This is the difference between `multiple` and `single` sets change in lean mass. This coefficient has an accompanying confidence interval that can be used for inference. Let's add the difference between groups as an additional point to the plot together with a confidence interval and a second scale.

```
descriptives <- delta_lean |>
  mutate(volume = factor(volume, levels = c("single", "multiple"),
                        labels = c("Low-volume", "Moderate-volume"))) |>
  summarise(.by = volume,
            m = mean(delta))

zero <- descriptives |>
  filter(volume == "Low-volume") |>
  pull(m)

est <- zero + (- mod_results[3,1])
ciu <- - mod_results[3,4] + zero
cil <- - mod_results[3,5] + zero

delta_lean |>
  mutate(volume = factor(volume, levels = c("single", "multiple"),
                        labels = c("Low-volume", "Moderate-volume"))) |>
  ggplot(aes(volume, delta,
```

①

②

```

    group = participant)) +

geom_point(data = filter(descriptives, volume== "Low-volume"), ①
           aes(volume, m, group = NULL, fill = NULL),
           position = position_nudge(x = -0.1)) +

                                           ③
geom_segment(y = zero, yend = zero, x = 0.95, xend = 2.2, color = "gray50",
            lty = 1) +

                                           ④
geom_errorbar(aes(y = est, ymin = cil,
                 ymax = ciu, x = 2.1),
             width = 0, color = "gray50") +

geom_point(data = filter(descriptives, volume== "Moderate-volume"), ②
           aes(volume, m, group = NULL, fill = NULL),
           position = position_nudge(x = 0.1),
           shape = 24,
           size = 2,
           fill = "gray60") +

geom_line(alpha = 0.5) +

labs(y = "Change in lean mass from pre- to post-intervention (g)") +

theme_minimal() +

theme(legend.position = "none",
      axis.title.x = element_blank(),
      panel.grid.major.x = element_blank(),
      panel.grid.minor.y = element_blank(),
      panel.grid.major.y = element_blank())

```

- ① A “zero line” is created from the descriptive data in the low-volume group
- ② The confidence interval is “corrected” to the observed data

- as the model CI only shows the differences between groups. Notice that estimates are reversed in the model.
- ③ A segment is added to represent the “zero line”, the mean in low-volume.
 - ④ An errorbar is used around the descriptive data for the moderate volume group.

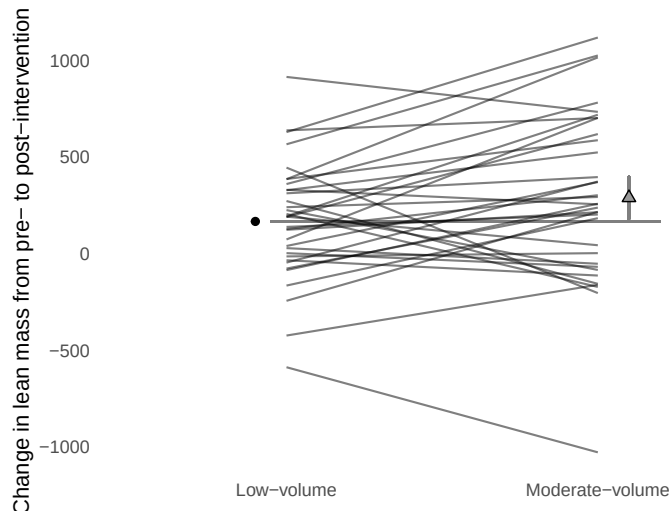


Figure 4.10: A line graph grouped by participant comparing conditions, reducing non-data ink with `theme_minimal` and removing redundant data ink by removing points and adding transparency. Adding group averages with `geom_point`.

The resulting graph shows the change from pre- to post-intervention in each condition per participant by lines. The average at the reference category represents a baseline to which the moderate volume condition is compared. Inference can be made from a 95% confidence interval around the moderate-volume condition.

4.1 Edit and revise in practice

The process of creating graphs is an iterative practice. We try something, edit and revise. When the end result goes into a

publication it is a good idea to have a workflow that updates the end result not some intermediate result.

Let us say that we are planning to submit a paper to the American Journal of Physiology. In the author instruction we read that a single column figure should be less or equal to 8.9 cm in width and a maximum 22.8 in height/depth.¹ This gives some basics instructions for our plot.

Graphs could be created from separate R scripts. This makes the workflow smoother as we only need to run a single script to recreate the output after revision. I keep my figures together with the scripts in a dedicated folder in my project folder called figures. Here I number figures and their corresponding scripts as `figure1.R`, `figure1.pdf`, `figure2.R` etc.

On the bottom of the script I have a call to the function `ggsave`. This function makes it easy to save a ggplot in a number of formats, also formats accepted by journals (such as TIF and PDF).

When I make changes to the script and saves them they are automatically sourced and the output is recreated.

4.2 Additional notes, tips and topics

4.2.1 Labels, annotations and special texts

A package called `ggtext` makes it possible to write markdown syntax in labels and annotation in ggplots. This improves usability to a large degree as subscripts and superscripts together with special characters are easily added to labels etc.

Sometimes custom fonts are needed. Your basic collection of fonts are not that impressive but packages such as `showtext` may help in installing additional fonts.

¹See <https://journals.physiology.org/manuscript-prep#figures>

```

library(showtext)

font_add_google(name = "Permanent Marker",
                family = "permanentmarker") ①
showtext_auto()

leg_leanmass |>
  filter(include != "excl") |>
  mutate(time = factor(time,
                        levels = c("pre", "post"))) |>

ggplot(aes(time, lean_mass, fill = volume)) +
  geom_boxplot() +

  labs(x = "Time",
       y = "Leg lean\ncmass (g)",
       fill = "Volume condition",
       title = "The plot title",
       subtitle = "The plot subtitle",
       caption = "the plot caption") +

  scale_x_discrete(labels = c("Pre\nintervention",
                             "Post\nintervention")) +

  theme(axis.title.x = element_blank(),
        legend.position = "top",
        plot.title = element_text(color = "black",
                                   family = "permanentmarker", ②
                                   size = 40))

```

- ① This “installs” the font on your system.
- ② Specifying a font family, use your own specified name to access it.

THE PLOT TITLE

The plot subtitle

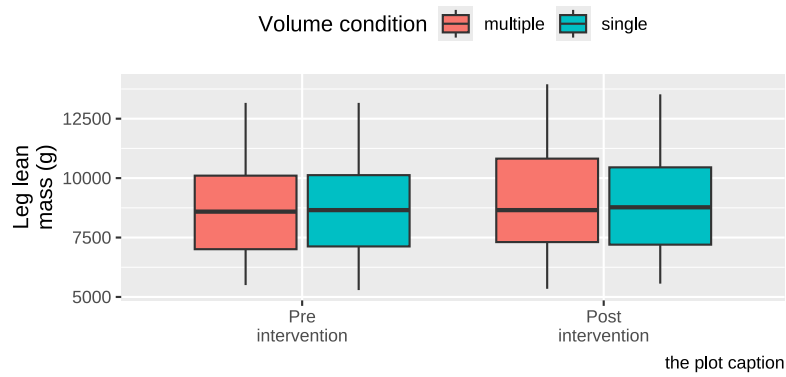


Figure 4.11: Adding a silly font to a basic graph

5 Glamorous graphics

In a [talk at RStudio conference in 2020](#), Will Chase presented principles related to glamorous graphics. In contrast to Tufte's principles (discussed above), glamorous graphics suggest adding ink to increase the attractiveness of graphics. The ideas presented by Chase might at first seem distant from dry scientific graphics, where no other message should be conveyed than what the data shows. However, good-looking graphics might communicate your message more effectively than bad-looking graphics, even though the style itself carries some additional message. Consider the plots below. What message other than the story related to the data do they convey?

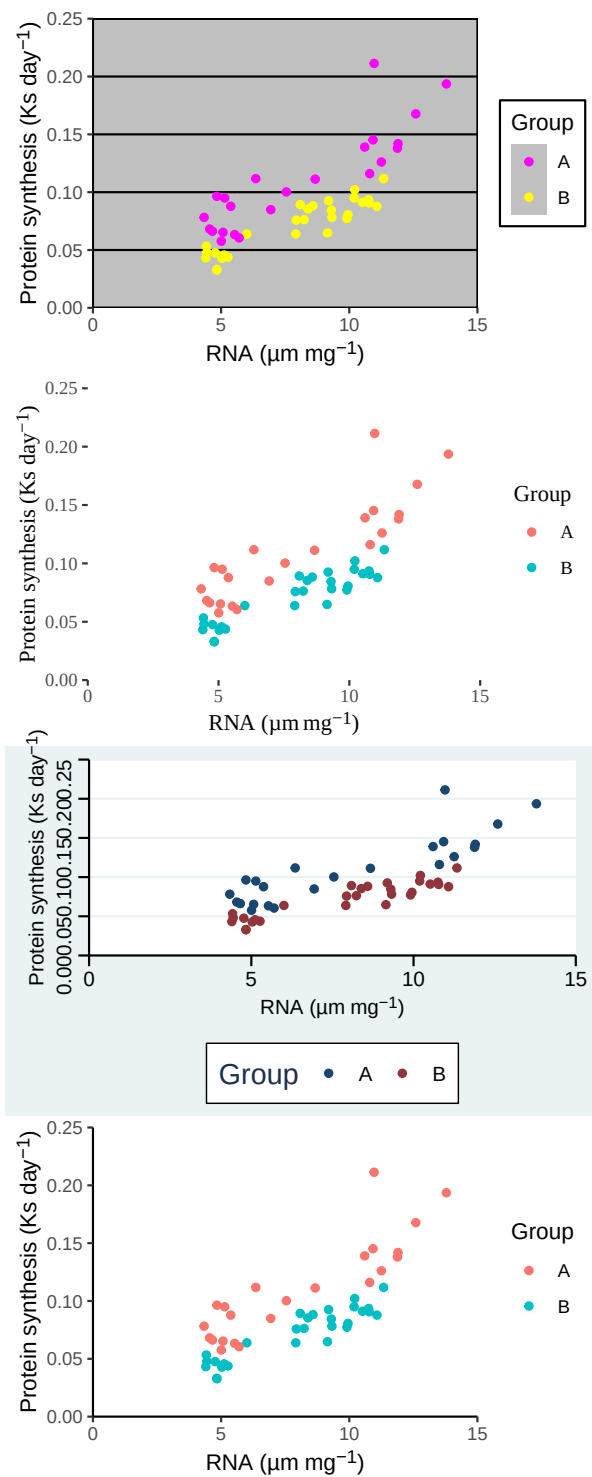


Figure 5.1: Different style choices also communicates messages.

Part II

Version control and collaborative scientific coding

Reproducibility and transparency are aspects of scientific research practices that have great potential to affect the quality of scientific results. Quality, in the sense that results will be available for scrutiny by researchers, funding agencies, and the public. As scientists, we have a moral obligation to be transparent and strive towards reproducibility, and to have these as goals in their own right will likely make us better scientists.

Version control software allows researchers to automate the process of keeping a record of changes in a project. In creating this record, we also create transparency and allow for reproducibility. As scientists, shifting our focus from a single end product, such as a scientific paper, to researching and communicating with transparency and reproducibility may alter our perspective on the scientific process.

Collaboration in complex projects is messy. Multiple files exist in various versions; changes are made in parallel with little to no control over what is lost or gained. Although most scientific collaborations would benefit from a more formal structure for collaboration on planning, data analysis, and writing, such structures are difficult to establish without a common point of departure. This part of our course will focus on tools and workflows that will make collaboration more effective. We will introduce version control as a tool in collaborative scientific writing and discuss how to establish a transparent framework for a collaborative writing project.

6 Introduction to git and GitHub

Git is a version control system that you install on your local machine. It helps you create a record of changes that you make to files in a specific repository where git is initialized. The git software stores earlier versions of your repository content in a hidden directory called `.git`.

Git works great for working with plain text files, such as `.txt`, `.md`, `.qmd`, `.csv`. With such files, git can provide you with an overview of changes or diffs that have occurred between two versions of the file. Other file types, such as `.docx` or `.xlsx`, are also tracked by git, but changes are not human-readable in the git system.

Git can be set up to communicate with an online repository. There are several alternatives for online hosting of version-controlled repositories, but GitHub is a popular alternative in the science community (Chen, Toro-Moreno, and Subramaniam 2025; Blischak, Davenport, and Wilson 2016; Ram 2013). GitHub offers online hosting and tools for collaboration.

To enable version control and collaboration, you must therefore have git installed and an active GitHub account.

6.0.1 Accounts and git-to-GitHub integration

Git can be installed from <https://git-scm.com>. Git requires no additional account information or software. To enable RStudio to act as a Git client, you must specify the location of your Git installation. This is done under *Tools > Global options > Git/SVN*.

Your local version history maintained by git can be controlled from your terminal window. However, to connect to GitHub's online services, you will need to set up git with your GitHub account. The easiest way to do this is to use the [GitHub CLI](#) which is a command-line interface to GitHub that makes it easy to authenticate and communicate with GitHub from your terminal. Install [GitHub CLI here](#) and run `gh auth login` in your terminal to set things up.

6.0.2 Local git edits

As already mentioned, git stores a record of changes to files in a given repository. A repository is a directory in which you have initialized git. We can do this in any directory using the command line by typing

```
git init
```

in our terminal. If you initialize a git (version control) project using RStudio it will run this command for you. You may check that you have initialized the version control system by typing

```
git status
```

You make decisions on what files to add to your version history and when to do so. A file, let us say `file-a.txt` is added to the staging area with the command

```
git add file-a.txt
```

The staging area is where files are kept until you commit a snapshot of them into the version history. We could also stage all files that have changes made to them by typing

```
git add -A
```

The next step is to commit changes. A commit is a snapshot of the repository or files at a given moment. We perform a commit after staging files.

```
git commit -m "my commit message"
```

The `-m "my commit message"` adds a short commit message to the commit. This should be a description of the changes made to the repository. A longer commit message can be added by only initializing a commit using `git commit`. This command will bring up a dialog in your terminal, allowing you to enter a short message on the first line. After leaving one blank line, a longer commit message can be added.

When a file has been committed to the version history, it is marked as *unmodified* until you make changes to the file. Modified files can again be staged (e.g. `git add <file>`) and committed.

Files that are tracked by mistake can be un-tracked without being removed using

```
git rm --cached <file>
```

This leaves the file in your local folder but removes it from the repository. Using `git rm <file>` removes the file from disc.

A local git “edit cycle” is shown in Figure 6.1.

```
library(ggplot2); library(ggtext)

ggplot(data.frame(x = c(0,1), y = c(0,1)), aes(x, y)) +

  scale_y_continuous(limits = c(0.3, 1)) +

  # Workspace
  geom_rect(aes(xmin = 0.01,
               xmax = 0.6,
               ymin = 0.35,
               ymax = 1),
            fill = "steelblue",
            alpha = 0.2) +

  geom_rect(aes(xmin = 0.02,
               xmax = 0.3,
```



```

    ymin = 0.35,
    ymax = 0.92),
    fill = "steelblue",
    alpha = 0.2) +

geom_rect(aes(xmin = 0.31,
    xmax = 0.58,
    ymin = 0.35,
    ymax = 0.92),
    fill = "steelblue",
    alpha = 0.2) +

    annotate("richtext",
    x = c(0.02, 0.32),
    hjust = 0,
    y = 0.89,
    label = c("*Untracked*", "*Tracked*"),
    fill = NA, label.color = NA) +

    annotate("richtext",
    x = 0.02,
    y = 0.97,
    label = "**Workspace**",
    hjust = 0,
    size = 8,
    fill = NA, label.color = NA, # remove background and outline
    label.padding = grid::unit(rep(0, 4), "pt")) + # remove padding) +

# Stage area
geom_rect(aes(xmin = 0.61,
    xmax = 1,
    ymin = 0.35,
    ymax = 1),
    fill = "purple",
    alpha = 0.2) +

    annotate("richtext",

```

```

    x = 0.62,
    y = 0.97,
    label = "**Staging area**",
    hjust = 0,
    size = 8,
    fill = NA, label.color = NA, # remove background and outline
    label.padding = grid::unit(rep(0, 4), "pt")) + # remove padding) +

## Labels and arrows

annotate("segment", y = c(0.85, 0.85),
            yend = c(0.85, 0.82),
            x = c(0.1, 0.9),
            xend = c(0.9, 0.9),
            arrow = arrow(length = unit(c(0, 2.5), "mm"), type = "closed")) +

geom_label(aes(x = 0.62, y = 0.85 , label = "<file>"),
            hjust = 0) +

geom_label(aes(x = 0.02, y = 0.85 , label = "Add file `git add <file>`"),
            hjust = 0) +

annotate("segment",
            y = c(0.8,0.75),
            yend = c(0.75, 0.75),
            x = c(0.9,0.9),
            xend = c(0.9, 0.48),
            arrow = arrow(length = unit(c(0, 2.5), "mm"), type = "closed")) +

annotate("segment",
            y = c(0.75, 0.7),

```

```

    yend = c(0.7, 0.7),
    x = c(0.35, 0.35),
    xend = c(0.35, 0.4),
    arrow = arrow(length = unit(c(0, 2.5), "mm"), type = "closed")) +

geom_label(aes(x = 0.65, y = 0.8 , label = "Commit file `git commit -m 'msg'`"),
  hjust = 0) +

geom_label(aes(x = 0.31, y = 0.75 , label = "Edit unmodified      "),
  hjust = 0) +

annotate("segment",
  y = c(0.7, 0.7),
  yend = c(0.7, 0.67),
  x = c(0.5, 0.9),
  xend = c(0.9, 0.9),
  arrow = arrow(length = unit(c(0, 2.5), "mm"), type = "closed")) +

geom_label(aes(x = 0.8, y = 0.7 , label = "<file>"),
  hjust = 0) +

geom_label(aes(x = 0.40, y = 0.7 , label = "Stage modified `git add <file>`"),
  hjust = 0) +

annotate("segment",
  y = c(0.65, 0.65),
  yend = c(0.65, 0.62),

```

```

        x = c(0.8, 0.35),
        xend = c(0.35, 0.35),
        arrow = arrow(length = unit(c(0, 2.5), "mm"), type = "closed")) +

geom_label(aes(x = 0.65, y = 0.65 , label = "Commit file `git commit -m 'msg'`"),
           hjust = 0) +

## Possible routs from unmodified

## Modify and commit
## Remove (untrack)
    annotate("segment",
            y = c(0.6, 0.6),
            yend = c(0.6, 0.54),
            x = c(0.4, 0.4),
            xend = c(0.70, 0.4),
            lty = 2,
            arrow = arrow(length = unit(c(2.5, 2.5), "mm"), type = "closed")) +
## File in staging area
    geom_label(aes(x = 0.75, y = 0.6 , label = "<file>"),
           hjust = 0.5) +

## Untrack file
    annotate("segment",
            y = c(0.5),
            yend = c(0.5),
            x = c(0.4),
            xend = c(0.1),
            lty = 2,
            arrow = arrow(length = unit(c(2.5), "mm"), type = "closed")) +

geom_label(aes(x = 0.31, y = 0.60 , label = "Unmodified      "),
           hjust = 0) +

```

```

## Remove file (delete)
  annotate("segment",
    y = c(0.5),
    yend = c(0.33),
    x = c(0.4),
    xend = c(0.4),
    lty = 2,
    arrow = arrow(length = unit(c(2.5), "mm"), type = "closed")) +

  annotate("label",
    x = 0.31, y = 0.5 ,
    label = "Untrack file\n`git rm --cached <file>`",

    hjust = 0) +

## File in trash
  geom_label(aes(x = 0.4, y = 0.30 , label = "<file>"),
    hjust = 0.5) +

## File untracked
  geom_label(aes(x = 0.06, y = 0.5 , label = "<file>"),
    hjust = 0.5) +

  geom_label(aes(x = 0.31, y = 0.40 , label = "Delete file from disc\n`git rm <file>`",
    hjust = 0)) +

theme_void()

```

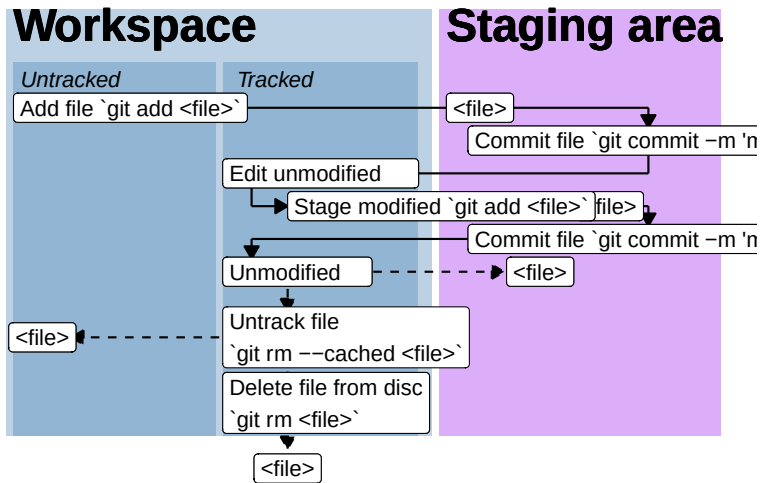


Figure 6.1: A cycle of local git edits. A files is added to the staging area by `git add <file>` and comitted to the *local repository* by `git commit`. The file is edited and is marked as modified after which it is staged (`git add <file>`). Again, staged modifications are committed and the file is once again marked as unmodified until edited. Files may be untracked from the repository or deleted from disc using `git rm` with or without the `--cached` option.

6.1 Local and online git usage

Your local git folder can be connected to an online repository. The online repository, or remote, is the repository in which you collaborate with others. The online repository can be set up as a starting point for your project or created from the command line in an already initialized git repository using GitHub CLI.

6.1.1 Starting with GitHub

After logging on to GitHub, select *New*, or go to github.com/new. This will bring you to a form that will help you create your repository. You need to give it a name, describe it, decide if you want to initialize with a README file (you want to do

that), and choose a license. Once all this is done, you will be able to copy the HTTPS address that you find under *Code* in your repository in GitHub. The HTTPS address is then entered in the *New Project Dialog* in RStudio after selecting *Version Control* under the different project types.

You have now initialized a remote repository and then cloned it to your computer as a local copy. You may now make changes to the project and add/commit these to your local version history.

6.1.2 Starting with a RStudio project

When starting a new RStudio project without version control, you have the possibility to initialize git in the dialog box. If you decide not to do this, the next step should be to write `git init` in your terminal to initialize git in your current project folder.

We can now create a remote repository using GitHub CLI with the command `gh repo create`. If this command is used without any subcommand, you will be guided through the process on the command line. If you have your project running and want to use it to create a repository on GitHub you should select “*Push an existing local directory to GitHub*”. You will be asked:

- The path to your local repository (defaults to `.`, the current directory)
- The name of the repository (defaults to your local folder name)
- If the repository should be private or public.
- A description of the repository (A short description of what the repository contains)
- If a remote should be added (Yes)
- What the remote should be called (Default to origin)

If successful, you will get a message telling you that the remote has been added. Your GitHub profile will now have a repository acting as the remote for your local repository.

6.2 Local to remote workflows

The basic workflow (Figure 6.2) for updating and downloading from the remote repository can include `git push` to push all local commits to the remote. This will update the remote with any changes you have committed to the version history. When the remote has been changed, you can download these changes in two ways. First, `git pull` downloads all changes and directly overwrites your local files, updating them to their latest versions. This also includes the removal of files and the addition of new files. Alternatively, you want to have more control over what is going into your local repository. In that case, you can use `git fetch`, which updates the local repository, but not your working directory.

To review changes made to the remote repository without merging, you can do `git log HEAD origin/main`, where “git log” is the basic command to inspect the latest commit message on the most recent version (HEAD) of the remote repository (“origin/main”). You could also inspect the actual change. Using `git diff HEAD origin/main`, we get an overview of all changes made to all files in the remote repository compared to the local repository. To do this for a specific file, for example, `README.md`, we can use `git diff HEAD origin/main -- README.md`.

Using `git merge`, after `git fetch`, you will merge downloaded versions of files from the remote with your working directory. `git fetch` and `git merge` can thus be a way to continue working on files while updating your repository with the latest changes. The shortcut for doing `git fetch` followed by `git merge` is `git pull`, as this command does both without the middle step.

After a `git pull`, we may still inspect what changed. Using `git diff`, we can compare the most current version to the parent commit, or the commit before the current version. To do this, we write `git diff HEAD~1`, which will give us a comparison between the “HEAD” (the current state of the repository) and “HEAD minus 1” (one commit behind the HEAD). Using `HEAD~2` will do the comparison with the commit two levels up from the HEAD.


```

ggplot(data.frame(x = c(0,1), y = c(0,1)), aes(x, y)) +

  scale_y_continuous(limits = c(0.3, 1)) +

  # Local working directory / Repository
  geom_rect(aes(xmin = 0.01,
                xmax = 0.6,
                ymin = 0.35,
                ymax = 1),
            fill = "steelblue",
            alpha = 0.2) +

  geom_rect(aes(xmin = 0.02,
                xmax = 0.3,
                ymin = 0.35,
                ymax = 0.92),
            fill = "steelblue",
            alpha = 0.2) +

  geom_rect(aes(xmin = 0.31,
                xmax = 0.58,
                ymin = 0.35,
                ymax = 0.92),
            fill = "steelblue",
            alpha = 0.2) +

  # Remote repository
  geom_rect(aes(xmin = 0.7,
                xmax = 0.95,
                ymin = 0.35,
                ymax = 1),
            fill = "steelblue",
            alpha = 0.2) +

  annotate("richtext",
          x = c(0.02, 0.32),
          hjust = 0,
          y = 0.89,
          label = c("*Working directory*", "*Local repository*"),

```

```

        fill = NA, label.color = NA) +

annotate("richtext",
  x = c(0.02, 0.7),
  y = c(0.97, 0.97),
  label = c("**Workspace**", "**Remote**"),
  hjust = 0,
  size = 8,
  fill = NA, label.color = NA) + # remove padding)

# Git add, commit
# git push
# git pull
# git fetch / merge
  annotate("segment",
    y = c(0.8, 0.7, 0.6, 0.5, 0.5),
    yend = c(0.8, 0.7, 0.6, 0.5, 0.5),
    x = c(0.1, 0.4, 0.8, 0.8, 0.4),
    xend = c(0.5, 0.8, 0.12, 0.5, 0.12),
    arrow = arrow(length = unit(c(2.5), "mm"), type = "closed")) +

  annotate("label",
    x = c(0.1, 0.4, 0.8, 0.8, 0.35),
    y = c(0.8, 0.7, 0.6, 0.5, 0.5),
    label = c("git add <file>\n git commit -m 'msg'",
              "git push",
              "git pull",
              "git fetch",
              "git merge"),

    hjust = 0) +

theme_void()

```

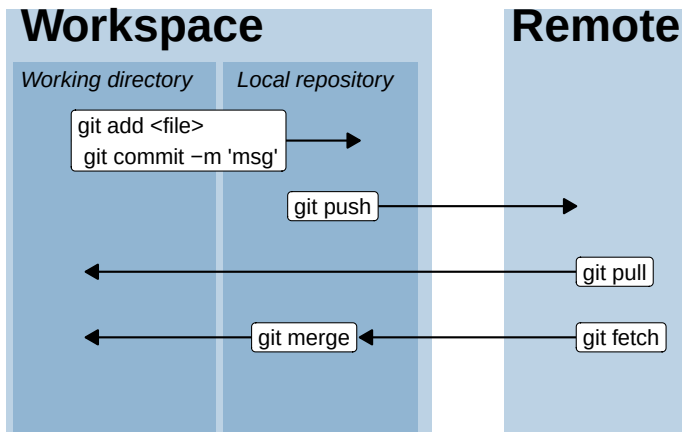


Figure 6.2: Local git commits can be pushed to a remote repository, this updates the remote with your local changes. If changes are made to the remote these can be pulled to the local repository and working directory. Git pull overwrites your local files in the working directory at the same time as updating the local version history. Git fetch downloads changes to your local repository without merging files. Git merge attempts to merge files in the working directory with the most recent changes from the remote repository.

6.2.1 Branches and pull requests

Sometimes changes to a repository can be expected to get big. Multiple files need updating, affecting numerous aspects of the repository. A large change to a project may break scripts or output in unexpected ways, so we want to ensure everything works before replacing the old with the new. Instead of implementing these changes incrementally in the main repository, we could create a new branch. A branch starts at a specific state of the repository and adds new history without affecting the main branch. When changes have been made to the branch, they can be incorporated into the main branch using a merge or as a pull request.

We can create a new branch using `git branch <new branch name>`, to create a new branch. This will create the branch but now switch to it. Switching between branches is done using `git checkout <branch name>`. If a branch is up to date with the main branch, it may be deleted (`git branch -d <branch name>`). If changes have been made, you will be asked if you really want to delete it. If we want to merge changes in the new branch with the main branch, we “checkout” the main branch (`git checkout main`), and merge with the other branch `git merge <branch name>`. After a merge, the branch can be safely deleted.

A branch can also be used in a pull request. To create a pull request, we first need to push the local new branch to the remote repository. If we have checked out the new branch, adding a new commit and are trying to push it to the remote, we will get a notification that current branch has no upstream branch. To push the local new branch to the remote we need to do `git push --set-upstream origin <branchname>`. This will create the new branch in the remote and push changes. A pull request is a feature of GitHub. To create pull requests on the command line, we will use GitHub CLI commands, starting with `gh pr create` (gh, activates GitHub CLI; pr, short for pull request; create, subcommand for initializing a pull request). This command will guide you through creating a pull request from the current branch, to the main branch. A pull request can be reviewed and merged online. GitHub provides a nice interface to inspect or discuss changes that are being made. After merging (or closing) the pull request we want to pull the latest updates to the local main and delete the new feature branch. We can delete the remote new feature using `git push origin --delete <branchname>`.

6.2.2 Forks and pull requests

Forks are copies of an online repository from one user to another user. This is useful when you want to contribute to a public repository but you do not have access to it. The basic workflow starts with creating a fork in the online GitHub interface. This will clone the repository to your user profile and make it easy

to download the repository to a local project. After making edits (commits), you can issue a pull request from your GitHub profile.

6.3 Finding help on the command line

In the above discussion, we are using the command line. This might feel scary. In a day to day workflow, however, you will use a small number of commands making it easy to remember them. If you need help, often, you can use `<command> --help` to get a list of commands.

6.4 More features in GitHub: Issues and Pages

Issues are a feature of GitHub that allows for creating notes on potential improvements, bugs, etc. They can be accessed and edited using GitHub CLI using `gh issue` or in the GitHub web interface. These can be a good way to track progress in a project and keep notes of choices made in the analysis/writing process.

GitHub allows us to create websites from a repository. These websites will be accessible for anyone and provide us with an opportunity to publish, for example, supplementary material in a nice format. Quarto can be used to create webpages directly from the repository.

7 A tentative workflow for scientific writing

The online remote repository may have several purposes in the context of writing a scientific paper. It will serve as a collaborative repository for you and your collaborators to work on. It can be used to showcase code and computations, thereby supporting your paper. It can also act as a backup and reference for yourself. As mentioned earlier, GitHub provides multiple tools that can make the writing process more effective in creating numerous outputs (a manuscript, a website, issue tracking, and more). In this chapter we will discuss a suggestion for a basic repository template for writing a scientific paper. We will also address writing workflows for two different scenarios, writing on your own, or in collaboration.

7.1 A basic template for an effective repository

Table 7.1 suggests a basic repository structure for a scientific writing project. The template aims to balance flexibility with generability. We want to be able to tweak the organization of the project if needed, but we also want to feel at home when switching between projects. The first two files in Table 7.1 are important for maintaining a clean workflow; the README file can be used to outline the project and list dependencies. Dependencies, in this context, could be software, data, or any other components needed to reproduce the analysis. The `.gitignore` file should be actively maintained to ensure that we version control only the files we want. For example, outputs should often be excluded from version control, as they can trigger merge conflicts. At some point in the project, we might want to add

output files to the online repository, as these are easier to read. At this point, we need to update `.gitignore` with new rules.

Table 7.1: Basic repository template for a scientific writing project

file-tree	description
<code>.gitignore</code>	An up to date registry of what you think git should ignore. Importantly, this will avoid trouble created by adding e.g., large files to the version history. You will also avoid merge conflicts due to changes in e.g., HTML files (output)
<code>README.md</code>	A README file used to give a synopsis of the project, describe the structure of the repository and any dependencies (software, data etc.) needed to reproduce the results.
<code>/data</code>	When analytic data is stored together with the repository, it goes into the data folder. This folder should be further subdivided.
<code>/data/raw-data</code>	Raw data goes into a specific folder, it is raw and might need processing before analysis.
<code>/data/derived-data</code>	Intermediate data processed from raw data, or data derived from analyses can be placed in a specific sub-folder. In principal, all data in the derived data folder is reproducible using scripts.

file-tree	description
<code>/R</code>	The R folder contains all scripts used to pre-process and analyse data.
<code>/figures/</code>	It is generally tidy practice to keep figure files in a separate folder.
<code>/resources</code>	Resources are additional files needed for compiling the manuscript file.
<code>/resources/citations-style.asx</code>	Citation Style Language (CSL) file contains the necessary information for Quarto to render a bibliography from citations entries.
<code>/resources/bibliography.bib</code>	A bibliography file contains all entries used for citation. Usually, this is a BibTeX file, editable as a plain text file.
<code>manuscript.qmd</code>	The manuscript file is the source file for the manuscript output file.

The data folder is subdivided into raw and derived data. The idea is to clearly separate collected data from data created as intermediate data in data cleaning or from, e.g., running statistical models. The derived data folder could, in principle, be deleted and recreated using scripts. The reason for not doing this when developing the analysis, though, is to avoid having to wait for analyses to complete.

The R folder contains all scripts needed to perform the analysis (except figures, see below). R scripts are used to import and clean data, make analyses (run models, clean output), and organize results. R files can be sourced in subsequent scripts and in the manuscript file. My suggestion for a clean repository is to keep figure source files in a separate folder together with the output from these files. We submit a scientific paper, the figures are often submitted as standalone files. Having a

dedicated subfolder for this work makes it easier to manage final editing. Figure source files can easily be sourced into the main manuscript file if we want to add them as part of the manuscript output.

The resources folder is used to collect all files needed to build the manuscript output. This usually starts with adding a Citation Style Language (CSL) file and a file containing the bibliography in BibTeX format. A CSL file relevant for the planned submission can often be found in the [Citation Style Language GitHub repository](#), which hosts > 1000 different styles for specific scientific journals. A bibliography file can be built using, e.g., your own Zotero library or online tools such as [doi2bib](#). RStudio also has integrated tools for working with citations, which can be used to update a maintained BibTeX file.

The manuscript file is the source file for the manuscript output. This is where we combine the text outline of the paper with R (or other language) code to produce the manuscript. Some analyses can be incorporated into the manuscript file, but heavy computations and data pre-processing might benefit from having dedicated R scripts (placed in the R folder).

7.2 A workflow for solo writing

After setting up a basic repository outline, the primary purpose of keeping a track of updates using the version control system should be to maintain a functional copy of the project. This means that changes committed to the repository history should work and, e.g., not break the rendering of output. This requires that the project be maintained by small incremental changes that are tested before being committed to the version history. An “informal” test could be to inspect the output of rendering the manuscript file. In more complex analyses, a set of tests can be incorporated into the project, ensuring that, for example, derived data are created with expected data types or dimensions, and that statistical models run correctly, etc. Such tests could be created programmatically (see below).

When an update to the project is satisfactory, a commit to the version history is made. By the end of the day, a push to the

remote repository makes it function as a backup. Additionally, the capabilities available at GitHub make it possible to keep notes on milestones and issues for the project.

7.2.1 Large revisions - Branches

When a writing project is due for a significant revision, maybe as a result of peer review, we might want to keep an unrevised copy of the project for reference. We can do this by using the branching functionality of git. A branch is a parallel version history branching from a specific time point in a project¹. In this context, we are using a branch to incorporate changes into the project without breaking the functionality in the main branch.

To create a new branch, we would use the command line and write `git branch revision`, where `revision` is the name of the new branch. This will create a copy of the main branch and initiate a separate version history for the specific branch. We can now incrementally add changes to the new branch and still have the functioning main branch as a reference or fallback. To switch between branches in our project, we use `git checkout revision`. A clean workflow could make use of GitHub issues together with branching to systematically address reviewer comments (as suggested by e.g., Van den Burg (2019)).

Git is designed to make branching easy and lightweight (Chacon 2014, 63). A branch can develop into a new branch and merge again with the parent branch. This allows for experimental work on a separate branch from the revision branch, enabling safe experimentation with potentially breaking changes to the project.

7.3 Writing in a team

Suppose you own a repository and want to invite collaborators to contribute to it. In that case, you need to add them to the

¹See [here](#) for a detailed overview.

project in GitHub. This is done under *Settings* and *Collaborators*. When working as a team, the remote repository becomes more important because it's where you get the latest version of the project at the start of a writing day. `git pull` incorporates the latest changes into your local repository, and pushing to the remote makes sure that your changes are made available to all other team members. It may be important to check for updates regularly when working on the same branch.

Using `git pull` regularly ensures that any merge conflicts can be resolved locally and that the fix is incorporated in your commit. A merge conflict occurs when changes are made to two versions of the same file that `git` cannot merge. When you encounter a merge conflict, you will need to manually fix it by choosing which version you want to keep. `git` flags merge conflict by a message similar to this:

```
$ git merge
Auto-merging manuscript.qmd
CONFLICT (content): Merge conflict in manuscript.qmd
Automatic merge failed; fix conflicts and then commit the result.
```

You can use `git status` to see which file is affected by the conflict. When opening the flagged file, you will encounter:

```
<<<<<<< HEAD
Text in your local version of the file
=====
Text in the remote version of the file
<<<<<<< REMOTE
```

By removing the whole segment (starting with `<<<<<<< HEAD`), and replacing it with a version you think should go into the file, you are resolving the conflict.

7.3.1 Branches and pull requests

Teamwork on writing could also benefit from creating pull requests from branches. A pull request is a formal request to incorporate changes made to a branch into the main (or any

selected) branch. When a collaborator has been working on a large change to the project as part of a branch, its incorporation could be reviewed as part of a pull request. GitHub makes it easy to discuss pull requests.

7.4 Doing formal testing

When writing an R package, the use of formal tests is a way to ensure that components that are part of the package (e.g., functions and data) work the way they are intended. Formal tests can be written, for example, to check if a data set used for analysis contains all expected rows. The toy example below evaluates the data frame stored in `d` and checks its dimensions. The “test” is silently passed if everything works, but returns an error if the test does not return all `TRUE`. This test could be incorporated as part of sanity checks in R scripts and the manuscript file, and is part of good practice when working on complex projects.

```
d <- data.frame(var1 = c(1, 2, 3),
                var2 = c("a", "b", "c"))

# Check if d is of correct dimensions
# This returns an error if not all expressions are evaluated as TRUE.
stopifnot({
  dim(d)[1] == 3
  dim(d)[2] == 2
})
```

More complicated tests can be added to the project. For this purpose, testing functions that are part of the [testthat package](#) could be helpful. In the context of writing tests for a paper we could use these testing functions as part of scripts, giving us a `Test passed` when the test was executed with expected results.

7.5 Going further - Make files

In the suggested workflow outlined at the beginning of this chapter, the manuscript file essentially works as a makefile. The makefile sources some R scripts to rerun analyses. However, when a data analysis grows complex, more formalism may be needed to ensure that things work out as intended. The [targets package](#) is an R package designed to create data analysis pipelines in R. The idea is that output is only generated when all components of a pipeline are up to date. **targets** supplies R-based tools to the user for the creation of data pipelines that could include data cleaning, modelling, etc.

8 Writing a paper using Quarto

8.1 The quarto publishing system

The quarto publishing system allows you to create reports, books, presentations, websites and blogs by converting computer code (like R) and markdown syntax to different output formats. In this chapter we will touch upon some features of this system that makes it easier to work with.

8.2 The basics - What Quarto does

Quarto is basically a software that converts source documents (`.qmd` files) to different output formats, like word, pdf or html. Figure 8.1. The conversion is made possible by `pandoc` which is a software that converts between different markup languages. A markup language is a system used to include structure and format in your plain text editor. We have already covered the basic for markdown (`?@sec-markdown`).



Figure 8.1

Quarto is the latest generation of software that has been created to make it easy to write technical reports combining data, code and text. Remember the discussion we had concerning reproducibility, (`?@sec-reproducibility`), quarto, and previous system, are designed for reproducible reporting.

When we work in a quarto file we basically write a computer program that gives us an output file. The output file is a report we can show to others, but we can also show how the output was created by showing the *source files*.

References

- Blischak, John D., Emily R. Davenport, and Greg Wilson. 2016. “A Quick Introduction to Version Control with Git and GitHub.” Edited by Francis Ouellette. *PLOS Computational Biology* 12 (1): e1004668. <https://doi.org/10.1371/journal.pcbi.1004668>.
- Chacon, Scott. 2014. *Pro Git*. 2nd ed. Berkeley, CA: Apress L. P.
- Chen, Katharine Y., Maria Toro-Moreno, and Arvind Rasi Subramaniam. 2025. “GitHub Enables Collaborative and Reproducible Laboratory Research.” *PLOS Biology* 23 (2): e3003029. <https://doi.org/10.1371/journal.pbio.3003029>.
- Gelman, Andrew, Cristian Pasarica, and Rahul Dodhia. 2002. “Let’s Practice What We Preach.” *The American Statistician* 56 (2): 121–30. <https://doi.org/10.1198/000313002317572790>.
- Hammarström, Daniel, Sjur Øfsteng, Lise Koll, Marita Hanes-tadhaugen, Ivana Hollan, William Apró, Jon Elling Whist, Eva Blomstrand, Bent R. Rønnestad, and Stian Ellefsen. 2020. “Benefits of higher resistance-training volume are related to ribosome biogenesis.” *The Journal of physiology* 598 (3): 543–65. <https://doi.org/10.1113/JP278455>.
- Ho, Jose, Tayfun Tunkaya, Sameer Aryal, Hyungwon Choi, and Adam Claridge-Chang. 2019. “Moving Beyond P Values: Data Analysis with Estimation Graphics.” *Nature Methods* 16 (7): 565–66. <https://doi.org/10.1038/s41592-019-0470-3>.
- Ram, Karthik. 2013. “Git Can Facilitate Greater Reproducibility and Increased Transparency in Science.” *Source Code for Biology and Medicine* 8 (1): 7. <https://doi.org/10.1186/1751-0473-8-7>.
- Spear, M. E. 1952. *Charting Statistics*. McGraw-Hill. <https://books.google.no/books?id=1vBOAAAAMAAJ>.

- Tufte, Edward R. 2001. *The Visual Display of Quantitative Information*. 2. edition. Cheshire, Conn.: Graphics Press.
- Tukey, John Wilder. 1977. *Exploratory Data Analysis*. Addison-Wesley Series in Behavioral Science. Reading, Mass: Addison-Wesley Pub. Co.
- Van den Burg, G. J. J. 2019. “Organizing a Paper Revision with GitHub.” *Gertjanvandenburgh.com/Blog*. https://gertjanvandenburgh.com/blog/github_paper_revision/.
- Wickham, Hadley. 2010. “A Layered Grammar of Graphics.” *Journal of Computational and Graphical Statistics* 19 (1): 3–28. <https://doi.org/10.1198/jcgs.2009.07098>.
- . 2014. “Tidy Data.” *Journal of Statistical Software; Vol 1, Issue 10 (2014)*, September. <https://www.jstatsoft.org/v059/i10> <http://dx.doi.org/10.18637/jss.v059.i10>.
- . 2016. *Ggplot2 : Elegant Graphics for Data Analysis*. 2nd ed. Use r! Cham: Springer International Publishing : Imprint: Springer. <https://doi.org/10.1007/978-3-319-24277-4>.
- Wilkinson, Leland, and Graham Wills. 2005. *The Grammar of Graphics*. 2nd ed. Statistics and Computing. New York: Springer.